



HAYABUSA

[English] | [[日本語](#)]

Stable Version [v3.6.0](#) GitHub Downloads [236k](#) GitHub Stars [2.9k](#) Contributors [19](#)

Black Hat Arsenal [Asia 2022](#) CODE BLUE Bluebox [2022](#) SECCON [2023](#) Security MiniCamp Tokyo [2023](#)

SANS DFIR Summit [2023](#) BSides Tokyo [2024](#) Hack Fes. [2024](#) HITCON [2024](#) SecTor [2024](#)

SINCON Kampung Workshop [2025](#) Black Hat Arsenal USA [2025](#) Maintenance Level [Actively Developed](#)

commits [4.9k](#)  [codecov](#) [81%](#)  Follow [@SecurityYamato](#)

About Hayabusa

Hayabusa is a **Windows event log fast forensics timeline generator** and **threat hunting tool** created by the [Yamato Security](#) group in Japan. Hayabusa means "[peregrine falcon](#)" in Japanese and was chosen as peregrine falcons are the fastest animal in the world, great at hunting and highly trainable. It is written in memory-safe [Rust](#), supports multi-threading in order to be as fast as possible and is the only open-source tool that has full support for the Sigma specification including v2 correlation rules. Hayabusa can handle parsing [upstream Sigma](#) rules, however, the Sigma rules that we use and host in the [hayabusa-rules repository](#) have some conversion done to them in order to make rule loading more flexible and reduce false positives. You can read the details about this at the [sigma-to-hayabusa-converter repository](#) README file. Hayabusa can be run either on single running systems for live analysis, by gathering logs from single or multiple systems for offline analysis, or by running the [Hayabusa artifact](#) with [Velociraptor](#) for enterprise-wide threat hunting and incident response. The output will be consolidated into a single CSV/JSON/JSONL timeline for easy analysis in [LibreOffice](#), [Timeline Explorer](#) [Elastic Stack](#), [Timesketch](#), etc...

Companion Projects

- [EnableWindowsLogSettings](#) - Documentation and scripts to properly enable Windows event logs.
- [Hayabusa Encoded Rules](#) - The same as Hayabusa Rules repository but the rules and config files are stored in one file and XORed to prevent false positives from anti-virus.
- [Hayabusa Rules](#) - Hayabusa and curated Sigma detection rules used Hayabusa.
- [Hayabusa EVTX](#) - A more maintained fork of the `evtx` crate.

- [Hayabusa Sample EVTxs](#) - Sample evtX files to use for testing hayabusa/sigma detection rules.
- [Presentations](#) - Presentations from talks that we have given about our tools and resources.
- [Sigma to Hayabusa Converter](#) - Curates upstream Windows event log based Sigma rules into an easier to use form.
- [Takajo](#) - An analyzer for hayabusa results.
- [WELA \(Windows Event Log Analyzer\)](#) - An analyzer for Windows event logs written in PowerShell. (Deprecated and replaced by Takajo.)

Third-Party Projects That Use Hayabusa

- [AllthingsTimesketch](#) - A NodeRED workflow that imports Plaso and Hayabusa results into Timesketch.
- [LimaCharlie](#) - Provides cloud-based security tools and infrastructure to fit your needs.
- [OpenRelik](#) - An open-source (Apache-2.0) platform designed to streamline collaborative digital forensic investigations.
- [Splunk4DFIR](#) - Quickly spin up a splunk instance with Docker to browse through logs and tools output during your investigations.
- [Velociraptor](#) - A tool for collecting host based state information using The Velociraptor Query Language (VQL) queries.

Table of Contents

- [About Hayabusa](#)
- [Companion Projects](#)
- [Third-Party Projects That Use Hayabusa](#)
 - [Table of Contents](#)
 - [Main Goals](#)
 - [Threat Hunting and Enterprise-wide DFIR](#)
 - [Fast Forensics Timeline Generation](#)
- [Screenshots](#)
 - [Startup](#)
 - [DFIR Timeline Terminal Output](#)
 - [Keyword Search Results](#)
 - [Detection Frequency Timeline \(-T option\)](#)
 - [Results Summary](#)
 - [HTML Results Summary \(-H option\)](#)
 - [DFIR Timeline Analysis in LibreOffice \(-M Multiline Output\)](#)
 - [DFIR Timeline Analysis in Timeline Explorer](#)
 - [Critical Alert Filtering and Computer Grouping in Timeline Explorer](#)
 - [Analysis with the Elastic Stack Dashboard](#)
 - [Analysis in Timesketch](#)
- [Importing and Analyzing Timeline Results](#)
- [Analyzing JSON-formatted results with JQ](#)
- [Features](#)
- [Downloads](#)
 - [Windows live response packages](#)
- [Git Cloning](#)
- [Advanced: Compiling From Source \(Optional\)](#)
 - [Updating Rust Packages](#)
 - [Cross-compiling 32-bit Windows Binaries](#)
 - [macOS Compiling Notes](#)
 - [Linux Compiling Notes](#)
 - [Cross-compiling Linux MUSL Binaries](#)
- [Running Hayabusa](#)
 - [Caution: Anti-Virus/EDR Warnings and Slow Runtimes](#)
 - [Windows](#)

- [Error when trying to scan a file or directory with a space in the path](#)
 - [Characters not being displayed correctly](#)
- [Linux](#)
- [macOS](#)
- [Command List](#)
 - [Analysis Commands:](#)
 - [Config Commands:](#)
 - [DFIR Timeline Commands:](#)
 - [General Commands:](#)
- [Command Usage](#)
 - [Analysis Commands](#)
 - [computer-metrics command](#)
 - [computer-metrics command examples](#)
 - [computer-metrics screenshot](#)
 - [eid-metrics command](#)
 - [eid-metrics command examples](#)
 - [eid-metrics command config file](#)
 - [eid-metrics screenshot](#)
 - [expand-list command](#)
 - [expand-list command examples](#)
 - [expand-list results](#)
 - [extract-base64 command](#)
 - [extract-base64 command examples](#)
 - [extract-base64 results](#)
 - [log-metrics command](#)
 - [log-metrics command examples](#)
 - [log-metrics screenshot](#)
 - [logon-summary command](#)
 - [logon-summary command examples](#)
 - [logon-summary screenshots](#)
 - [pivot-keywords-list command](#)
 - [pivot-keywords-list command examples](#)
 - [pivot-keywords-list config file](#)
 - [search command](#)
 - [search command examples](#)
 - [search command config files](#)
 - [Config Commands](#)
 - [config-critical-systems command](#)
 - [config-critical-systems command examples](#)
 - [DFIR Timeline Commands](#)
 - [Scan Wizard](#)
 - [Core Rules](#)
 - [Core+ Rules](#)
 - [Core++ Rules](#)
 - [Emerging Threats \(ET\) Add-On Rules](#)
 - [Threat Hunting \(TH\) Add-On Rules](#)
 - [Channel-based event log and rules filtering](#)
 - [csv-timeline command](#)
 - [csv-timeline command examples](#)
 - [Advanced - GeolP Log Enrichment](#)

- [GeolP config file](#)
 - [Automatic updates of GeolP databases](#)
 - [csv-timeline command config files](#)
 - [json-timeline command](#)
 - [json-timeline command examples and config files](#)
 - [level-tuning command](#)
 - [level-tuning command examples](#)
 - [level-tuning config file](#)
 - [list-profiles command](#)
 - [set-default-profile command](#)
 - [set-default-profile command examples](#)
 - [update-rules command](#)
 - [update-rules command example](#)
- [Timeline Output](#)
 - [Output Profiles](#)
 - [1. minimal profile output](#)
 - [2. standard profile output](#)
 - [3. verbose profile output](#)
 - [4. all-field-info profile output](#)
 - [5. all-field-info-verbose profile output](#)
 - [6. super-verbose profile output](#)
 - [7. timesketch-minimal profile output](#)
 - [8. timesketch-verbose profile output](#)
 - [Profile Comparison](#)
 - [Profile Field Aliases](#)
 - [Extra Profile Field Alias](#)
 - [Abbreviations](#)
 - [Level Abbreviations](#)
 - [MITRE ATT&CK Tactics Abbreviations](#)
 - [Channel Abbreviations](#)
 - [Other Abbreviations](#)
 - [Progress Bar](#)
 - [Color Output](#)
 - [Results Summary](#)
 - [Detection Fequency Timeline](#)
- [Hayabusa Rules](#)
 - [Sigma v.s. Hayabusa \(Built-in Sigma Compatible\) Rules](#)
- [Other Windows Event Log Analyzers and Related Resources](#)
- [Windows Logging Recommendations](#)
- [Sysmon Related Projects](#)
- [Community Documentation](#)
 - [English](#)
 - [Japanese](#)
- [Contribution](#)
- [Bug Submission](#)
- [License](#)
- [Twitter](#)

Main Goals

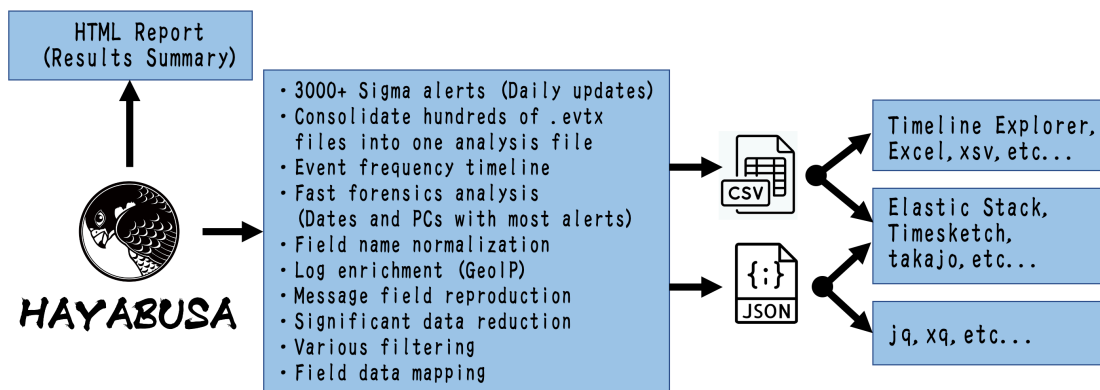
Threat Hunting and Enterprise-wide DFIR

Hayabusa currently has over 4000 Sigma rules and over 170 Hayabusa built-in detection rules with more rules being added regularly. It can be used for enterprise-wide proactive threat hunting as well as DFIR (Digital Forensics and Incident Response) for free with [Velociraptor's Hayabusa artifact](#). By combining these two open-source tools, you can essentially retroactively reproduce a SIEM when there is no SIEM setup in the environment. You can learn about how to do this by watching [Eric Capuano's Velociraptor walkthrough](#) [here](#).

Fast Forensics Timeline Generation

Windows event log analysis has traditionally been a very long and tedious process because Windows event logs are 1) in a data format that is hard to analyze and 2) the majority of data is noise and not useful for investigations. Hayabusa's goal is to extract out only useful data and present it in a concise as possible easy-to-read format that is usable not only by professionally trained analysts but any Windows system administrator. Hayabusa hopes to let analysts get 80% of their work done in 20% of the time when compared to traditional Windows event log analysis.

DFIR Timeline Creation



Screenshots

Startup



by Yamato Security

Setting new standards in Windows event visibility~

Start time: 2024/11/25 10:14

Total event log files: 598

Total file size: 139.2 MB

Loading detection rules. Please wait.

Excluded rules: 26

Noisy rules: 12 (Disabled)

Deprecated rules: 216 (5.00%) (Disabled)

Experimental rules: 375 (8.69%)

Stable rules: 241 (5.58%)

Test rules: 3,700 (85.73%)

Unsupported rules: 42 (0.97%) (Disabled)

Correlation rules: 3 (0.07%)

Correlation referenced rules: 3 (0.07%)

Hayabusa rules: 175

Sigma rules: 4,141

Total detection rules: 4,316

Creating the channel filter. Please wait.

Evtx files loaded after channel filter: 585

Detection rules enabled after channel filter: 4,239

Output profile: standard

Scanning in progress. Please wait.

[00:00:07] 41 / 585 " [==>

] 7%

"../hayabusa-sample-evtx/EVTX-ATTACK-SAMPLES/AutomatedTestingTools/PanacheSysmon_vs_AtomicRedTeam01.evtx" (1.1 MB)

DFIR Timeline Terminal Output

```

2021-10-27 19:34:50.024 +09:00 · Mimikatz Use · high · FS03.offsec.lan · Sec · 4674 · 127533 · Svc: \REGISTRY\USER\S-1-5-21-4230534742-2542757381-3142984815-1111\Printers\Connections\,fs03vuln,Kiwi Legit Printer ! User: admmig ! AccessMask: DELETE_READ_CONTROL_WRITE_DAC_WRITE_OWNER_QUERY_KEY_VALUE_SET_KEY_VALUE_CREATE_SUBKEY_ENUMERATE_SUBKEYS_KEY_CHANGES_NOTIFICATION_CREATE_LINK ! LID: 0x123550 · AccessMask: DELETE_READ_CONTROL_WRITE_DAC_WRITE_OWNER_QUERY_KEY_VALUE_SET_KEY_VALUE_CREATE_SUBKEY_ENUMERATE_SUBKEYS_KEY_CHANGES_NOTIFICATION_CREATE_LINK ! HandleId: 0x6a4 ! ObjectServer: Security ! ObjectType: Key ! PrivilegeList: SeTakeOwnershipPrivilege ! ProcessId: 1048 ! ProcessName: C:\Windows\System32\spoolsv.exe ! SubjectDomainName: OFFSEC ! SubjectUserSid: S-1-5-21-4230534742-2542757381-3142984815-1111

2021-10-27 19:35:56.899 +09:00 · Proc Exec · info · FS03.offsec.lan · Sec · 4688 · 127567 · Cmdline: C:\Windows\system32\DllHost.exe /Processid:{AB8902B4-09CA-4BB6-B78D-A8F59079A8D5} ! Proc: C:\Windows\System32\dlh.exe ! PID: 3848 ! User: FS03$ ! LID: 0x3e7 · ProcessId: 564 ! SubjectDomainName: OFFSEC ! SubjectUserSid: S-1-5-18 ! TargetDomainName: OFFSEC ! TargetLogonId: 0x12377a ! TargetUserName: admmig ! TargetUserSid: S-1-5-21-4230534742-2542757381-3142984815-1111 ! TokenElevationType: LIMITED_TOKEN

2021-10-27 19:35:56.899 +09:00 · Susp CmdLine (Possible LOLBIN) · low · FS03.offsec.lan · Sec · 4688 · 127567 · Cmdline: C:\Windows\system32\DllHost.exe /Processid:{AB8902B4-09CA-4BB6-B78D-A8F59079A8D5} ! Proc: C:\Windows\System32\dlh.exe ! PID: 3848 ! User: FS03$ ! LID: 0x3e7 · ProcessId: 564 ! SubjectDomainName: OFFSEC ! SubjectUserSid: S-1-5-18 ! TargetDomainName: OFFSEC ! TargetLogonId: 0x12377a ! TargetUserName: admmig ! TargetUserSid: S-1-5-21-4230534742-2542757381-3142984815-1111 ! TokenElevationType: LIMITED_TOKEN

2021-10-28 22:41:21.325 +09:00 · Unusual Parent Process For Cmd.EXE · med · FS03.offsec.lan · Sysmon · 1 · 144552 · Cmdline: "cmd.exe" ! Proc: C:\Windows\System32\cmd.exe ! User: NT AUTHORITY\SYSTEM ! ParentCmdline: C:\Windows\System32\spoolsv.exe ! LID: 0x3e7 ! LGUID: 7CF65FC7-7BF5-6176-E703-000000000000 ! PID: 3388 ! PGUID: 7CF65FC7-A881-617A-0605-000000001300 ! ParentPID: 1048 ! ParentPGUID: 7CF65FC7-7BF9-6176-1A00-000000001300 ! Description: Windows Command Processor ! Product: Microsoft® Windows® Operating System ! Company: Microsoft Corporation ! Hashes: SHA1=7C3D7281E1151FE4127923F4B4C3CD36438E1A12_MD5=F5AE03DE0AD60F5B17B82F2CD68402FE_SHA256=6F88FB88FFB0F1D5465C2826E5B4F5235988B1B8377C8378FFEBEC171BAD18B_IMPHASH=77AED1ADAF248344F08C8AD1432908C3 · CurrentDirectory: C:\Windows\system32\ ! FileVersion: 6.3.9600.17415

```

Keyword Search Results

```

2022-02-23 19:10:12.152 +09:00 · Admin login · dc1.test.local · Sec · 4672 · 12557 · PrivilegeList: SeSecurityPrivilege SeBackupPrivilege SeRestorePrivilege SeTakeOwnershipPrivilege SeDebugPrivilege SeSystemEnvironmentPrivilege SeLoadDriverPrivilege SeImpersonatePrivilege SeDelegateSessionUserImpersonatePrivilege SeEnableDelegationPrivilege ! SubjectDomainName: TEST ! SubjectLogonId: 0x86d96c ! SubjectUserName: DC1$ ! SubjectUserSid: S-1-5-18 · ../hayabusa-sample-evtx/YamatoSecurity/LateralMovement/T1210_Zerologon_WindowsExtraLogging.evtx

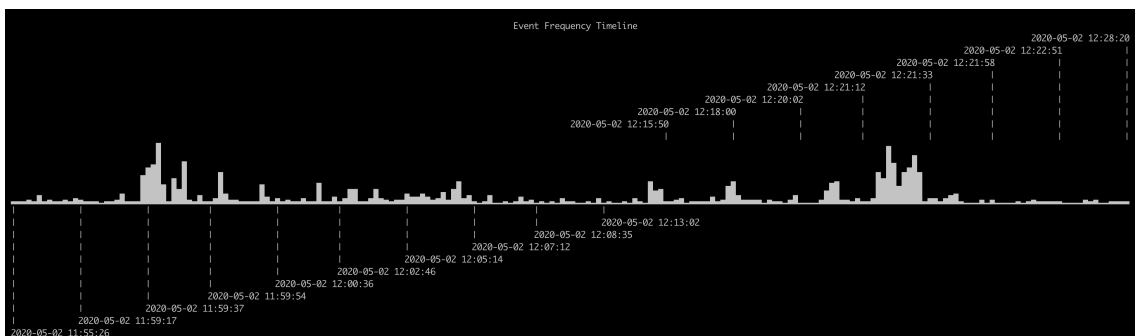
2022-02-23 19:10:22.852 +09:00 · Account logoff · dc1.test.local · Sec · 4634 · 12560 · LogonType: 3 ! TargetDomainName: TEST ! TargetLogonId: 0x86d96c ! TargetUserName: DC1$ ! TargetUserSid: S-1-5-18 · ../hayabusa-sample-evtx/YamatoSecurity/LateralMovement/T1210_Zerologon_WindowsExtraLogging.evtx

2022-02-23 19:11:06.073 +09:00 · Logon success · dc1.test.local · Sec · 4624 · 12562 · AuthenticationPackageName: Kerberos ! ElevatedToken: %%1842 ! ImpersonationLevel: %%1833 ! IpAddress: ::1 ! IpPort: 51377 ! KeyLength: 0 ! LmPackageName: - ! LogonGuid: F24A1AA9-4919-497F-5382-C96E5878DBFC ! LogonProcessName: Kerberos ! LogonType: 3 ! ProcessId: 0x0 ! ProcessName: - ! RestrictedAdminMode: - ! SubjectDomainName: - ! SubjectLogonId: 0x0 ! SubjectUserName: - ! SubjectUserSid: S-1-0-0 ! TargetDomainName: TEST.LOCAL ! TargetLinkId: 0x0 ! TargetLogonId: 0x871f5e ! TargetOutboundDomainName: - ! TargetOutboundUserName: - ! TargetUserName: DC1$ ! TargetUserSid: S-1-5-18 ! TransmittedServices: - ! VirtualAccount: %%1843 ! WorkstationName: - ../hayabusa-sample-evtx/YamatoSecurity/LateralMovement/T1210_Zerologon_WindowsExtraLogging.evtx

2022-02-23 19:11:06.073 +09:00 · Admin login · dc1.test.local · Sec · 4672 · 12561 · PrivilegeList: SeSecurityPrivilege SeBackupPrivilege SeRestorePrivilege SeTakeOwnershipPrivilege SeDebugPrivilege SeSystemEnvironmentPrivilege SeLoadDriverPrivilege SeImpersonatePrivilege SeDelegateSessionUserImpersonatePrivilege SeEnableDelegationPrivilege ! SubjectDomainName: TEST ! SubjectLogonId: 0x871f5e ! SubjectUserName: DC1$ ! SubjectUserSid: S-1-5-18 · ../hayabusa-sample-evtx/YamatoSecurity/LateralMovement/T1210_Zerologon_WindowsExtraLogging.evtx

```

Detection Frequency Timeline (-T option)



Results Summary

Results Summary:

Events with hits / Total events: 19,830 / 46,495 (Data reduction: 26,665 events (57.35%))

Total | Unique detections: 32,395 | 672
Total | Unique emergency detections: 15 (0.05%) | 4 (0.00%)
Total | Unique critical detections: 1,316 (4.06%) | 110 (9.08%)
Total | Unique high detections: 5,347 (16.51%) | 252 (9.52%)
Total | Unique medium detections: 2,401 (7.41%) | 181 (26.93%)
Total | Unique low detections: 5,017 (15.49%) | 64 (37.50%)
Total | Unique informational detections: 18,299 (56.49%) | 61 (16.37%)

Dates with most total detections:
emergency: 2020-08-02 (6), critical: 2021-11-03 (794), high: 2016-09-20 (3,654), medium: 2021-11-03 (733), low: 2016-09-20 (3,708), informational: 2016-08-19 (2,115)

Top 5 computers with most unique detections:
emergency: rootdcl.offsec.lan (2), srvdefender01.offsec.lan (2), FS03.offsec.lan (1), fs01.offsec.lan (1), fs03vuln.offsec.lan (1)
critical: IEWIN7 (61), FS03.offsec.lan (26), fs03vuln.offsec.lan (23), rootdcl.offsec.lan (14), srvdefender01.offsec.lan (13)
high: MSEDGWIN10 (102), IEWIN7 (58), FS03.offsec.lan (28), IE10Win7 (24), fs03vuln.offsec.lan (24)
medium: MSEDGWIN10 (91), FS03.offsec.lan (21), IEWIN7 (21), fs03vuln.offsec.lan (17), IE10Win7 (16)
low: MSEDGWIN10 (38), IE10Win7 (12), LAPTOP-JU4MB10E (9), jump01.offsec.lan (8), win10-02.offsec.lan (7)
informational: IEWIN7 (18), MSEDGWIN10 (17), PC01.example.corp (15), IE8Win7 (14), fs01.offsec.lan (14)

Top emergency alerts:	Top critical alerts:
Mimikatz DC Sync (5) Active Directory Replication from Non Machine Account (3) Potential SystemNightmare Exploitation Attempt (2) HackTool - Mimikatz Kirbi File Creation (1) Potential SMB Relay Attack Tool Execution (1)	Suspicious Service Path (246) Suspicious Service Installation Script (244) PowerShell Scripts Installed as Services (244) Malicious PowerShell Scripts - PoshModule (64) Suspicious Service Name (62)
Top high alerts:	Top medium alerts:
Metasploit SMB Authentication (3,561) Proc Injection (100) Remote Thread Creation Via PowerShell (93) Remote Thread Creation In Uncommon Target Image (93) Log File Cleared (87)	Possible LOLBIN (611) Non Interactive PowerShell Process Spawned (261) Potentially Malicious PwSh (171) Proc Access (91) User with Privileges Logon (80)
Top low alerts:	Top informational alerts:
Lagon Failure (Wrong Password) (3,575) Possible LOLBIN (807) User with Privileges Logon (99) CMD Shell Output Redirect (84) DLL Loaded (Sysmon Alert) (71)	Proc Exec (11,173) NetShare File Access (2,558) PwSh Scriptblock (789) PwSh Pipeline Exec (680) NetShare Access (403)

HTML Results Summary (-H option)



General Overview

- Start time: 2022/10/02 03:16
- Excluded rules: 12
- Noisy rules: 5 (Disabled)
- Experimental rules: 2036 (67.24%)
- Stable rules: 213 (7.03%)
- Test rules: 779 (25.73%)
- Hayabusa rules: 138
- Sigma rules: 2890
- Total enabled detection rules: 3028
- Elapsed Time: 00:00:23.844

Results Summary

- Saved alerts and events: 19,545
 - Total events analyzed: 76,967
 - Data reduction: 57,422 events (74.61%)
 - Dates with most total detections:
 - critical: 2019-07-19 (15)
 - high: 2016-09-20 (3,656)
 - medium: 2019-05-19 (165)
 - low: 2016-09-20 (3,780)
 - informational: 2016-08-19 (2,105)
- 

Computers with most unique critical detections:

- MSEDGEWIN10 (6)
- IEWIN7 (3)
- srvdefender01.offsec.lan (2)
- FS03.offsec.lan (2)
- rootdc1.offsec.lan (2)
- alice.insecurebank.local (1)
- IE10Win7 (1)
- DC1.insecurebank.local (1)
- win10-02.offsec.lan (1)
- fs01.offsec.lan (1)
- fs03vuln.offsec.lan (1)
- DESKTOP-PIU87N6 (1)

Computers with most unique high detections:

- MSEDGEWIN10 (112)
- IEWIN7 (72)

Top critical alerts:

- [Sticky Key Like Backdoor Usage](#) (10)
- [Meterpreter or Cobalt Strike Getsystem Service Installation](#) (6)
- [Active Directory Replication from Non Machine Account](#) (6)
- [Windows Defender Alert](#) (4)
- [WannaCry Ransomware](#) (4)
- [Dumpept Process Dumper](#) (3)
- [SystemNightmare Exploitation Script Execution](#) (2)
- [Mimikatz MemSSP Default Log File Creation](#) (2)
- [Suspicious LSASS Process Clone](#) (2)
- [TrustedPath UAC Bypass Pattern](#) (2)
- [SMB Relay Attack Tools](#) (1)
- [smbexec.py Service Installation](#) (1)
- [Malicious Named Pipe](#) (1)
- [CobaltStrike Service Installations in Registry](#) (1)
- [Lsass Memory Dump via Comsvcs DLL](#) (1)

Top high alerts:

- [Metasploit SMB Authentication](#) (3,562)
- [Malicious Svc Possibly Installed](#) (271)

DFIR Timeline Analysis in LibreOffice (-M Multiline Output)

A	C	D	E	G	H	
Timestamp	Chan	Even	Le	RuleTitle	RuleAuthor	Details
2019-08-05 18:39	Sec	4624	med	Pass the Hash Activity 2	Dave Kennedy Jeff Warren (method) David Vassallo (rule)	Type: 9 TgtUser: IEUser SrcComp: - SrcIP: :1 LID: 0x3887e
2019-08-05 18:39	Sec	4624	high	Successful Overpass the Hash Attempt	Roberto Rodriguez (source) Dominik Schaudel (rule)	Type: 9 TgtUser: IEUser SrcComp: - SrcIP: :1 LID: 0x3887e
2019-08-14 20:53	Sysmon	1	info	Proc Exec	Zach Mathis	Cmd: "C:\Windows\explorer.exe" shell...{7699427-3cc6-4b62-be14-2a705115b7ab} Proc: C:\Windows\explorer.exe User: MSEDGWIN10IEUser ParentCmd: C:\Windows\Explorer.EXE LID: 0x29126 PID: 1052 PGUID: 747F3D96-F639-5D53-0000-001067DA2600
2019-08-14 20:53	Sysmon	1	info	Proc Exec	Zach Mathis	Cmd: C:\Windows\explorer.exe factory.{75dff2b7-6936-4c06-a8bb-676a7b00b24b} -Embedding Proc: C:\Windows\explorer.exe User: MSEDGWIN10IEUser ParentCmd: C:\Windows\system32\svchost.exe -k DcomLaunch -p LID: 0x29126 PID: 6000 PGUID: 747F3D96-F639-5D53-0000-001092EE2600
2019-08-14 20:53	Sysmon	1	med	Explorer Process Tree Break	Florian Roth (Nextron Systems) Nasreddine Bensherchali (Nextron Systems) @gott_cyber	Cmd: C:\Windows\explorer.exe factory.{75dff2b7-6936-4c06-a8bb-676a7b00b24b} -Embedding Proc: C:\Windows\explorer.exe User: MSEDGWIN10IEUser ParentCmd: C:\Windows\system32\svchost.exe -k DcomLaunch -p LID: 0x29126 PID: 6000 PGUID: 747F3D96-F639-5D53-0000-001092EE2600
2019-08-14 20:53	Sysmon	1	info	Proc Exec	Zach Mathis	Cmd: "c:\windows\system32\wscript.exe" /E:vbs c:\windows\temp\icon.ico "powershell -exec bypass -c ""IE"" ([System.Text Proc: C:\Windows\System32\wscript.exe User: MSEDGWIN10IEUser ParentCmd: C:\Windows\explorer.exe /factory.{75dff2b7-6936-4c06-a8bb-676a7b00b24b} -Embedding LID: 0x29126 PID: 8180 PGUID: 747F3D96-F639-5D53-0000-0010B0FC2600
2019-08-14 20:53	Sysmon	1	med	Change PowerShell Policies to an Insecure Level	frack113	Cmd: "c:\windows\system32\wscript.exe" /E:vbs c:\windows\temp\icon.ico "powershell -exec bypass -c ""IE"" ([System.Text Proc: C:\Windows\System32\wscript.exe User: MSEDGWIN10IEUser ParentCmd: C:\Windows\explorer.exe /factory.{75dff2b7-6936-4c06-a8bb-676a7b00b24b} -Embedding LID: 0x29126 PID: 8180 PGUID: 747F3D96-F639-5D53-0000-0010B0FC2600
2019-08-14 20:53	Sysmon	1	high	PowerShell Base64 Encoded IEX Keyword	Florian Roth (Nextron Systems)	Cmd: "c:\windows\system32\wscript.exe" /E:vbs c:\windows\temp\icon.ico "powershell -exec bypass -c ""IE"" ([System.Text Proc: C:\Windows\System32\wscript.exe User: MSEDGWIN10IEUser ParentCmd: C:\Windows\explorer.exe /factory.{75dff2b7-6936-4c06-a8bb-676a7b00b24b} -Embedding LID: 0x29126 PID: 8180

DFIR Timeline Analysis in Timeline Explorer

Time	Computername	Eventid	Level	Alert	Details
2021-05-22 05:43:18.227 +09:00	fs01.offsec.lan	4648	informational	Explicit Logon	Source User: F501\$: Target User: admmig
2021-05-22 05:43:22.562 +09:00	fs01.offsec.lan	4625	low	Logon Failure - Wrong Password	User: admmig@offsec.lan : Type: 8 : Won
2021-05-22 05:43:49.345 +09:00	fs01.offsec.lan	4625	low	Logon Failure - Wrong Password	User: admmig@offsec.lan : Type: 8 : Won
2021-05-22 05:43:50.131 +09:00	fs01.offsec.lan	4625	low	Logon Failure - Wrong Password	User: admmig@offsec.lan : Type: 8 : Won
2021-05-22 05:43:50.607 +09:00	fs01.offsec.lan	4625	low	Logon Failure - Wrong Password	User: admmig@offsec.lan : Type: 8 : Won
2021-05-22 05:43:50.866 +09:00	fs01.offsec.lan	4625	low	Logon Failure - Wrong Password	User: admmig@offsec.lan : Type: 8 : Won
2021-05-23 06:56:57.685 +09:00	fs01.offsec.lan	1102	high	Security log was cleared	User: admmig
2021-05-23 06:57:11.842 +09:00	fs01.offsec.lan	4688	high	Relevant Anti-Virus Event	
2021-05-23 06:57:11.842 +09:00	fs01.offsec.lan	4688	critical	Mimikatz Use	
2021-05-26 22:02:27.149 +09:00	mssql01.offsec.lan	4624	informational	Logon Type 3 - Network	User: admmig : Workstation: - : IP Addr
2021-05-26 22:02:27.155 +09:00	mssql01.offsec.lan	5145	medium	DCERPC SMB Spoolss Named Pipe	
2021-05-26 22:02:27.155 +09:00	mssql01.offsec.lan	5145	critical	CVE-2021-1675 Print Spooler Exploitation IPC Access	
2021-05-26 22:02:29.726 +09:00	mssql01.offsec.lan	4624	informational	Logon Type 3 - Network	User: admmig : Workstation: - : IP Addr
2021-05-26 22:02:29.734 +09:00	mssql01.offsec.lan	5145	medium	DCERPC SMB Spoolss Named Pipe	
2021-05-26 22:02:29.734 +09:00	mssql01.offsec.lan	5145	critical	CVE-2021-1675 Print Spooler Exploitation IPC Access	
2021-05-26 22:02:34.373 +09:00	mssql01.offsec.lan	4624	informational	Logon Type 3 - Network	User: admmig : Workstation: - : IP Addr
2021-05-26 22:02:34.375 +09:00	mssql01.offsec.lan	5145	medium	DCERPC SMB Spoolss Named Pipe	
2021-05-26 22:02:34.379 +09:00	mssql01.offsec.lan	4624	informational	Logon Type 3 - Network	User: admmig : Workstation: - : IP Addr
2021-05-26 22:02:34.379 +09:00	mssql01.offsec.lan	4624	informational	Logon Type 3 - Network	User: admmig : Workstation: - : IP Addr
2021-05-26 22:02:34.380 +09:00	mssql01.offsec.lan	4624	informational	Logon Type 3 - Network	User: admmig : Workstation: - : IP Addr
2021-05-27 05:24:46.570 +09:00	rootdc1.offsec.lan	4768	medium	Possible AS-REP Roasting	Possible AS-REP Roasting
2021-05-27 05:24:46.570 +09:00	rootdc1.offsec.lan	4768	informational	Kerberos TGT was requested	User: admin-test : Service: krbtgt : IP
2021-06-01 23:06:34.542 +09:00	fs01.offsec.lan	4720	medium	Local user account created	User: WADGUtilityAccount : SID:S-1-5-21-1
2021-06-01 23:08:21.225 +09:00	fs01.offsec.lan	4720	medium	Local user account created	User: elie : SID:S-1-5-21-1081258321-3780
2021-06-03 21:17:56.988 +09:00	fs01.offsec.lan	1102	high	Security log was cleared	User: admmig
2021-06-03 21:18:12.941 +09:00	fs01.offsec.lan	4672	informational	Admin Logon	User: admmig : LogonID: 0x322e5b7
2021-06-03 21:18:12.942 +09:00	fs01.offsec.lan	4624	informational	Logon Type 3 - Network	User: admmig : Workstation: - : IP Addr
2021-06-04 03:34:12.672 +09:00	fs01.offsec.lan	4104	high	Windows Firewall Profile Disabled	
2021-06-04 04:17:44.873 +09:00	fs01.offsec.lan	1102	high	Security log was cleared	User: admmig

Critical Alert Filtering and Computer Grouping in Timeline Explorer

Computername					
Line	Tag	Time	Eventid	Level	Alert
▼	=	🕒	🕒	= critical	🕒
▶ Computername: 01566s-win16-ir.threebeesco.com (Count: 1)					
▶ Computername: alice.insecurebank.local (Count: 3)					
▶ ▶ Computername: DC1.insecurebank.local (Count: 18)					
5540	■	2019-03-26 06:28:45.026 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5539	■	2019-03-26 06:28:45.026 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5538	■	2019-03-26 06:28:45.026 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5537	■	2019-03-26 06:28:45.026 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5536	■	2019-03-26 06:28:45.025 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5535	■	2019-03-26 06:28:45.025 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5534	■	2019-03-26 06:28:45.025 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5533	■	2019-03-26 06:28:45.025 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5532	■	2019-03-26 06:28:45.025 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5531	■	2019-03-26 06:28:45.024 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5530	■	2019-03-26 06:28:45.024 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5529	■	2019-03-26 06:28:45.024 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5528	■	2019-03-26 06:28:45.023 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5527	■	2019-03-26 06:28:45.023 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5526	■	2019-03-26 06:28:45.023 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5525	■	2019-03-26 06:28:45.023 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5524	■	2019-03-26 06:28:45.022 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
5523	■	2019-03-26 06:28:45.022 +09:00	5136	critical	Powerview Add-DomainObjectAcl DCSync AD Extend Right
▶ Computername: DESKTOP-PIU87N6 (Count: 1)					

Analysis with the Elastic Stack Dashboard

 Elastic Stack Dashboard 1

 Elastic Stack Dashboard 2

Analysis in Timesketch

2019-05-08T02:10:43	☐ ★ 🔍 📄	Active Directory Replication from Non Machine Account	User: Administrator ObjSvr: DS ObjName: %c6fa700-bfe4-452a-a766-424f84c29583 OpType: Object Access HID: 0x0 LID: 0x40c6511	4662	DC1.insecurebank.local	Sec	T1003.006
2019-05-08T02:10:43	☐ ★ 🔍 📄	Active Directory Replication from Non Machine Account	User: Administrator ObjSvr: DS ObjName: %c6fa700-bfe4-452a-a766-424f84c29583 OpType: Object Access HID: 0x0 LID: 0x40c6511	4662	DC1.insecurebank.local	Sec	T1003.006
2019-05-08T02:10:43	☐ ★ 🔍 📄	Active Directory Replication from Non Machine Account	User: Administrator ObjSvr: DS ObjName: %c6fa700-bfe4-452a-a766-424f84c29583 OpType: Object Access HID: 0x0 LID: 0x40c6511	4662	DC1.insecurebank.local	Sec	T1003.006
4 days							
2019-05-12T12:52:43	☐ ★ 🔍 📄	Meterpreter or Cobalt Strike Getsystem Service Installation	Svc: WinPwnage Path: %COMSPEC% /c ping -n 1 127.0.0.1 >nul && echo 'WinPwnage' > \\.\pipe\WinPwnagePipe Acct: LocalSystem StartType: demand start	7045	IEWIN7	Sys	T1134.001 : T1134.002
39 days							
2019-06-21T07:35:37	☐ ★ 🔍 📄	Dumprt Process Dumper	Path: C:\Windows\Temp\dumprt.dmp Process: C:\Users\administrator\Desktop\x64\Outflank-Dumprt.exe PID: 3572 PGUID: ECAD0485-88C9-5D0C-0000-0010348C1D00	11	alice.insecurebank.local	Sysmon	T1003.001

Importing and Analyzing Timeline Results

You can learn how to analyze CSV timelines in Timeline Explorer [here](#).

You can learn how to import CSV files into Elastic Stack [here](#).

You can learn how to import CSV files into Timesketch [here](#).

Analyzing JSON-formatted results with JQ

You can learn how to analyze JSON-formatted results with `jq` [here](#).

Features

- Cross-platform support: Windows, Linux, macOS.
- Developed in Rust to be memory safe and fast.
- Multi-thread support delivering up to a 5x speed improvement.
- Creates single easy-to-analyze timelines for forensic investigations and incident response.
- Threat hunting based on IoC signatures written in easy to read/create/edit YML based hayabusa rules.
- Sigma rule support to convert sigma rules to hayabusa rules.
- Currently it supports the most sigma rules compared to other similar tools and even supports count rules and new aggregators such as `|equalsfield` and `|endswithfield`.
- Computer metrics. (Useful for filtering on/out certain computers with a large amount of events.)
- Event ID metrics. (Useful for getting a picture of what types of events there are and for tuning your log settings.)
- Rule tuning configuration by excluding unneeded or noisy rules.
- MITRE ATT&CK mapping of tactics.
- Rule level tuning.

- Create a list of unique pivot keywords to quickly identify abnormal users, hostnames, processes, etc... as well as correlate events.
- Output all fields for more thorough investigations.
- Successful and failed logon summary.
- Enterprise-wide threat hunting and DFIR on all endpoints with [Velociraptor](#).
- Output to CSV, JSON/JSONL and HTML Summary Reports.
- Daily Sigma rule updates.
- Support for JSON-formatted log input.
- Log field normalization. (Converting multiple fields with different naming conventions into the same field name.)
- Log enrichment by adding GeoIP (ASN, city, country) information to IP addresses.
- Search all events for keywords or regular expressions.
- Field data mapping. (Ex: `0xc0000234` -> `ACCOUNT LOCKED`)
- Evtx record carving from evtx slack space.
- Event de-duplication when outputting. (Useful when recovery records is enabled or when you include backed up evtx files, evtx files from VSS, etc...)
- Scan setting wizard to help choose which rules to enable easier. (In order to reduce false positives, etc...)
- PowerShell classic log field parsing and extraction.
- Low memory usage. (Note: this is possible by not sorting results. Best for running on agents or big data.)
- Filtering on Channels and Rules for the most efficient performance.
- Detect, extract and decode Base64 strings found in logs.
- Alert level adjustment based on critical systems.

Downloads

Please download the latest stable version of Hayabusa with compiled binaries or compile the source code from the [Releases](#) page.

We provide binaries for the following architectures:

- Linux ARM 64-bit GNU (`hayabusa-x.x.x-lin-aarch64-gnu`)
- Linux Intel 64-bit GNU (`hayabusa-x.x.x-lin-x64-gnu`)
- Linux Intel 64-bit MUSL (`hayabusa-x.x.x-lin-x64-musl`)
- macOS ARM 64-bit (`hayabusa-x.x.x-mac-aarch64`)
- macOS Intel 64-bit (`hayabusa-x.x.x-mac-x64`)
- Windows ARM 64-bit (`hayabusa-x.x.x-win-aarch64.exe`)
- Windows Intel 64-bit (`hayabusa-x.x.x-win-x64.exe`)
- Windows Intel 32-bit (`hayabusa-x.x.x-win-x86.exe`)

[For some reason the Linux ARM MUSL binary does not run properly](#) so we do not provide that binary. It is out of our control, so we plan on providing it in the future when it gets fixed.

Windows live response packages

As of v2.18.0, we are provide special Windows packages that use XOR-encoded rules provided in a single file as well as all of the config files combined into a single file (hosted at the [hayabusa-encoded-rules repository](#)). Just download the zip packages with `live-response` in the name. The zip files just include three files: the Hayabusa binary, XOR-encoded rules file and the config file. The purpose of these live response packages are for when running Hayabusa on client endpoints, we want to make sure that anti-virus scanners like Windows Defender do not give false positives on `.yaml` rule files. Also, we want to minimize the amount of files being written to the system so that forensics artifacts like the USN Journal do not get overwritten.

Git Cloning

You can `git clone` the repository with the following command and compile binary from source code:

Warning: The main branch of the repository is for development purposes so you may be able to access new features not yet officially released, however, there may be bugs so consider it unstable.

```
git clone https://github.com/Yamato-Security/hayabusa.git --recursive
```

Note: If you forget to use `--recursive` option, the `rules` folder, which is managed as a git submodule, will not be cloned.

You can sync the `rules` folder and get latest Hayabusa rules with `git pull --recurse-submodules` or use the following command:

```
hayabusa.exe update-rules
```

If the update fails, you may need to rename the `rules` folder and try again.

*Caution: When updating, rules and config files in the `rules` folder are replaced with the latest rules and config files in the [hayabusa-rules](#) repository. Any changes you make to existing files will be overwritten, so we recommend that you make backups of any files that you edit before updating. If you are performing level tuning with `level-tuning`, please re-tune your rule files after each update. If you add **new** rules inside of the `rules` folder, they will **not** be overwritten or deleted when updating.*

Advanced: Compiling From Source (Optional)

If you have Rust installed, you can compile from source with the following command:

Note: To compile, you usually need the latest version of Rust.

```
cargo build --release
```

You can download the latest unstable version from the main branch or the latest stable version from the [Releases](#) page.

Be sure to periodically update Rust with:

```
rustup update stable
```

The compiled binary will be outputted in the `./target/release` folder.

Updating Rust Packages

You can update to the latest Rust crates before compiling:

```
cargo update
```

Please let us know if anything breaks after you update.

Cross-compiling 32-bit Windows Binaries

You can create 32-bit binaries on 64-bit Windows systems with the following:

```
rustup install stable-i686-pc-windows-msvc
rustup target add i686-pc-windows-msvc
rustup run stable-i686-pc-windows-msvc cargo build --release
```

Warning: Be sure to run `rustup install stable-i686-pc-windows-msvc` whenever there is a new stable version of Rust as `rustup update stable` will not update the compiler for cross compiling and you may receive build errors.

macOS Compiling Notes

If you receive compile errors about openssl, you will need to install [Homebrew](#) and then install the following packages:

```
brew install pkg-config
brew install openssl
```

Linux Compiling Notes

The following build dependencies are required:

- openssl-devel (Fedora-based) / libssl-dev (Ubuntu-based)
- perl
- musl-gcc

Cross-compiling Linux MUSL Binaries

On a Linux OS, first install the target.

```
rustup install stable-x86_64-unknown-linux-musl
rustup target add x86_64-unknown-linux-musl
```

Compile with:

```
cargo build --release --target=x86_64-unknown-linux-musl
```

Warning: Be sure to run `rustup install stable-x86_64-unknown-linux-musl` whenever there is a new stable version of Rust as `rustup update stable` will not update the compiler for cross compiling and you may receive build errors.

The MUSL binary will be created in the `./target/x86_64-unknown-linux-musl/release/` directory. MUSL binaries are about 15% slower than the GNU binaries, however, they are more portable across different versions and distributions of linux.

Running Hayabusa

Caution: Anti-Virus/EDR Warnings and Slow Runtimes

You may receive an alert from anti-virus or EDR products when trying to run hayabusa or even just when downloading the `.yaml` rules as there will be keywords like `mimikatz` and suspicious PowerShell commands in the detection signature. These are false positives so will need to configure exclusions in your security products to allow hayabusa to run. If you are worried about malware or supply chain attacks, please check the hayabusa source code and compile the binaries yourself.

You may experience slow runtime especially on the first run after a reboot due to the real-time protection of Windows Defender. You can avoid this by temporarily turning real-time protection off or adding an exclusion to the hayabusa runtime directory. (Please take into consideration the security risks before doing these.)

Windows

In a Command/PowerShell Prompt or Windows Terminal, just run the appropriate 32-bit or 64-bit Windows binary.

Error when trying to scan a file or directory with a space in the path

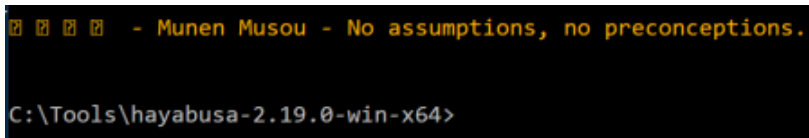
When using the built-in Command or PowerShell prompt in Windows, you may receive an error that Hayabusa was not able to load any .evtx files if there is a space in your file or directory path. In order to load the .evtx files properly, be sure to do the following:

1. Enclose the file or directory path with double quotes.
2. If it is a directory path, make sure that you do not include a backslash for the last character.

Characters not being displayed correctly

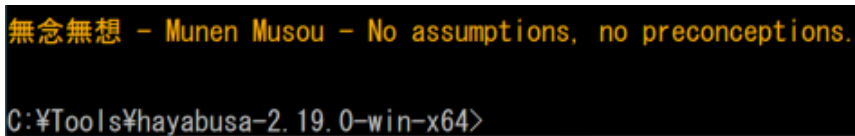
With the default font `Lucida Console` on Windows, various characters used in the logo and tables will not be displayed properly. You should change the font to `Consolas` to fix this.

This will fix most of the text rendering except for the display of Japanese characters in the closing messages:



You have four options to fix this:

1. Use [Windows Terminal](#) instead of the Command or PowerShell prompt. (Recommended)
2. Use the `MS Gothic` font. Note that backslashes will turn into Yen symbols.



3. Install the [HackGen](#) fonts and use `HackGen Console NF`.
4. Use the `-q, --quiet` to not display the closing messages that contain Japanese.

Linux

You first need to make the binary executable.

```
chmod +x ./hayabusa
```

Then run it from the Hayabusa root directory:

```
./hayabusa
```

macOS

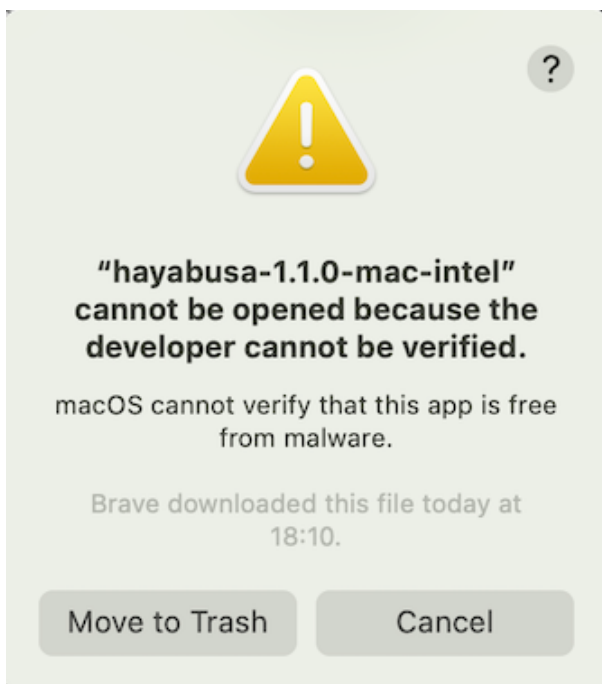
From Terminal or iTerm2, you first need to make the binary executable.

```
chmod +x ./hayabusa
```

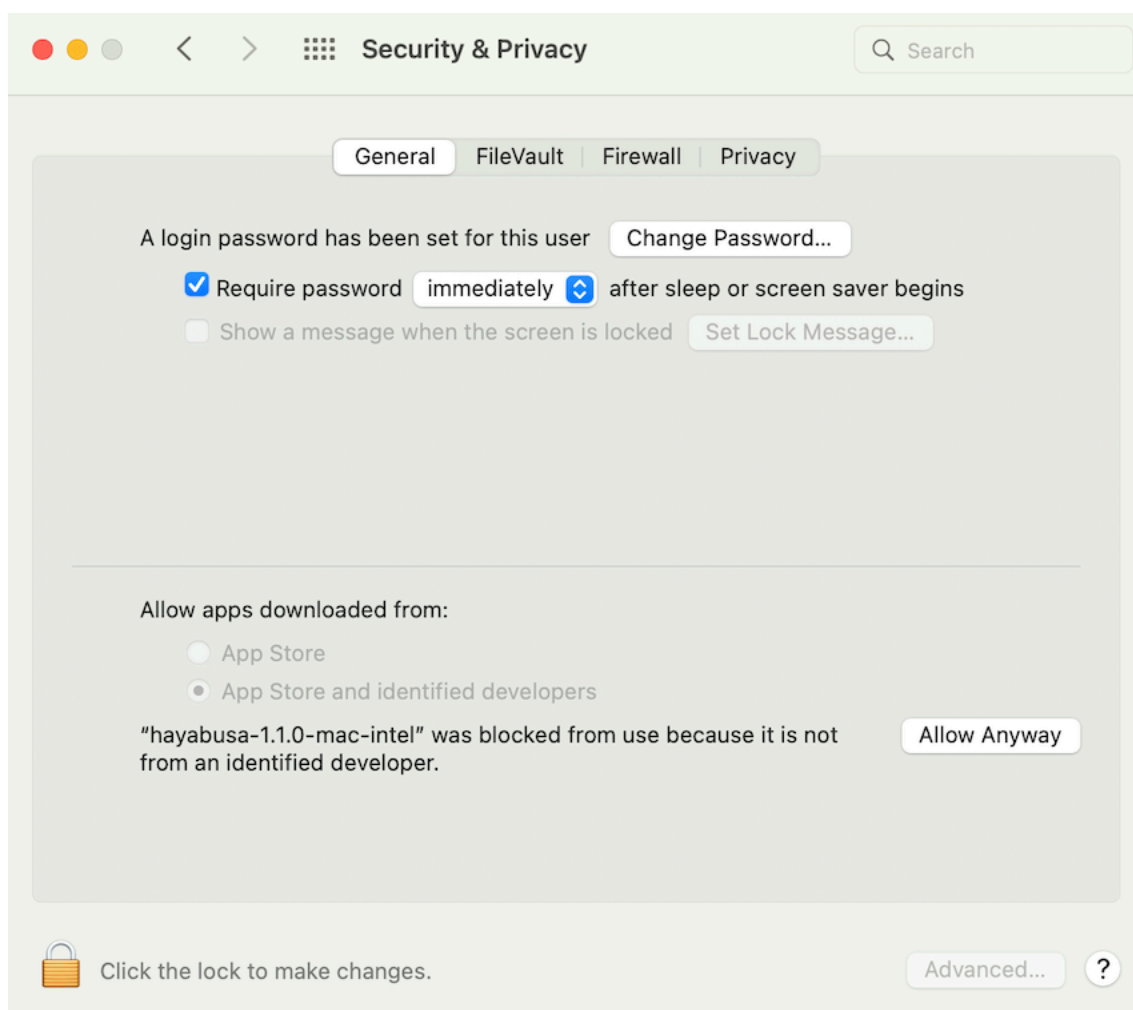
Then, try to run it from the Hayabusa root directory:

```
./hayabusa
```

On the latest version of macOS, you may receive the following security error when you try to run it:



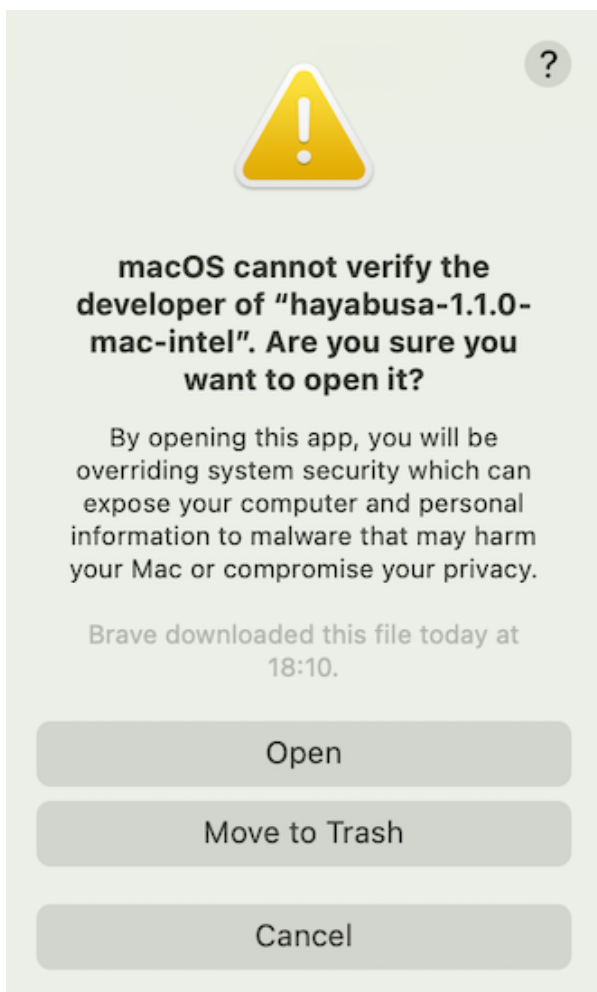
Click "Cancel" and then from System Preferences, open "Security & Privacy" and from the General tab, click "Allow Anyway".



After that, try to run it again.

```
./hayabusa
```

The following warning will pop up, so please click "Open".



You should now be able to run hayabusa.

Command List

Analysis Commands:

- `computer-metrics` : Print the number of events based on computer names.
- `eid-metrics` : Print the number and percentage of events based on Event ID.
- `expand-list` : Extract `expand` placeholders from the `rules` folder.
- `extract-base64` : Extract and decode base64 strings from events.
- `log-metrics` : Print log file metrics.
- `logon-summary` : Print a summary of logon events.
- `pivot-keywords-list` : Print a list of suspicious keywords to pivot on.
- `search` : Search all events by keyword(s) or regular expressions

Config Commands:

- `config-critical-systems` : Find critical systems like domain controllers and file servers.

DFIR Timeline Commands:

- `csv-timeline` : Save the timeline in CSV format.
- `json-timeline` : Save the timeline in JSON/JSONL format.

- `level-tuning` : Custom tune the alerts' `level` .
- `list-profiles` : List the available output profiles.
- `set-default-profile` : Change the default profile.
- `update-rules` : Sync the rules to the latest rules in the [hayabusa-rules](https://github.com/hayabusa-rules) GitHub repository.

General Commands:

- `help` : Print this message or the help of the given subcommand(s)
- `list-contributors` : Print the list of contributors

Command Usage

Analysis Commands

`computer-metrics` command

You can use the `computer-metrics` command to check how many events there are according to each computer defined in the `<System><Computer>` field. Be aware that you cannot completely rely on the `Computer` field for separating events by their original computer. Windows 11 will sometimes use completely different `Computer` names when saving to event logs. Also, Windows 10 will sometimes record the `Computer` name in all lowercase. This command does not use any detection rules so will analyze all events. This is a good command to run to quickly see which computers have the most logs. With this information, you can then use the `--include-computer` or `--exclude-computer` options when creating your timelines to make your timeline generation more efficient by creating multiple timelines according to computer or exclude events from certain computers.

Usage: `computer-metrics <INPUT> [OPTIONS]`

Input:

- d, --directory <DIR> Directory of multiple .evtx files
- f, --file <FILE> File path to one .evtx file
- l, --live-analysis Analyze the local C:\Windows\System32\winevt\Logs folder

General Options:

- C, --clobber Overwrite files when saving
- h, --help Show the help menu
- J, --JSON-input Scan JSON formatted logs instead of .evtx (.json or .jsonl)
- Q, --quiet-errors Quiet errors mode: do not save error logs
- x, --recover-records Carve evtx records from slack space (default: disabled)
- c, --rules-config <DIR> Specify custom rule config directory (default: ./rules/config)
- target-file-ext <FILE-EXT...> Specify additional evtx file extensions (ex: evtx_data)
- t, --threads <NUMBER> Number of threads (default: optimal number for performance)

Filtering:

- time-offset <OFFSET> Scan recent events based on an offset (ex: 1y, 3M, 30d, 24h, 30m)

Output:

- o, --output <FILE> Save the results in CSV format (ex: computer-metrics.csv)

Display Settings:

- K, --no-color Disable color output
- q, --quiet Quiet mode: do not display the launch banner
- v, --verbose Output verbose information

computer-metrics command examples

- Print computer name metrics from a directory: `hayabusa.exe computer-metrics -d ../logs`
- Save results to a CSV file: `hayabusa.exe computer-metrics -d ../logs -o computer-metrics.csv`

computer-metrics screenshot

Computer	Events
DESKTOP-6D0DBMB	1,755,914
Agamemnon	1,445,328
DESKTOP-A8CALR3	783,185
WIN-FPV0DSIC906.sigma.fr	292,717
WIN-TKC15D7KHUR	242,133
WIN-FPV0DSIC906	159,217
evtx-PC	129,528
WIN-06FB45IHQ35	5,555
WIN-KP04DDU11AB	2,133
WIN-VR474TJ38H3	971
37L4247D28-05	500

eid-metrics command

You can use the `eid-metrics` command to print out the total number and percentage of event IDs (`<System>` `<EventID>` field) separated by channels. This command does not use any detection rules so will scan all events.

Usage: `eid-metrics <INPUT> [OPTIONS]`

Input:

`-d, --directory <DIR>` Directory of multiple .evtx files
`-f, --file <FILE>` File path to one .evtx file
`-l, --live-analysis` Analyze the local C:\Windows\System32\winevt\Logs folder

General Options:

`-C, --clobber` Overwrite files when saving
`-h, --help` Show the help menu
`-J, --JSON-input` Scan JSON formatted logs instead of .evtx (.json or .jsonl)
`-Q, --quiet-errors` Quiet errors mode: do not save error logs
`-x, --recover-records` Carve evtx records from slack space (default: disabled)
`-c, --rules-config <DIR>` Specify custom rule config directory (default: ./rules/config)
`-t, --threads <NUMBER>` Number of threads (default: optimal number for performance)
`--target-file-ext <FILE-EXT...>` Specify additional evtx file extensions (ex: evtx_data)

Filtering:

`--exclude-computer <COMPUTER...>` Do not scan specified computer names (ex: ComputerA) (ex: ComputerA,ComputerB)
`--include-computer <COMPUTER...>` Scan only specified computer names (ex: ComputerA) (ex: ComputerA,ComputerB)
`--time-offset <OFFSET>` Scan recent events based on an offset (ex: 1y, 3M, 30d, 24h, 30m)

Output:

`-b, --disable-abbreviations` Disable abbreviations
`-o, --output <FILE>` Save the Metrics in CSV format (ex: metrics.csv)

Display Settings:

`-K, --no-color` Disable color output
`-q, --quiet` Quiet mode: do not display the launch banner
`-v, --verbose` Output verbose information

Time Format:

`--European-time` Output timestamp in European time format (ex: 22-02-2022 22:00:00.123+02:00)
`-O, --ISO-8601` Output timestamp in original ISO-8601 format (ex: 2022-02-22T10:10:10.1234567Z) (Always UTC)
`--RFC-2822` Output timestamp in RFC 2822 format (ex: Fri, 22 Feb 2022 22:00:00-0600)
`--RFC-3339` Output timestamp in RFC 3339 format (ex: 2022-02-22 22:00:00.123456-06:00)
`--US-military-time` Output timestamp in US military time format (ex: 02-22-2022 22:00:00.123 -06:00)
`--US-time` Output timestamp in US time format (ex: 02-22-2022 10:00:00.123 PM -06:00)
`-U, --UTC` Output time in UTC format (default: local time)

eid-metrics command examples

- Print Event ID metrics from a single file: `hayabusa.exe eid-metrics -f Security.evtx`

- Print Event ID metrics from a directory: `hayabusa.exe eid-metrics -d ../logs`
- Save results to a CSV file: `hayabusa.exe eid-metrics -f Security.evtx -o eid-metrics.csv`

`eid-metrics` command config file

The channel, event IDs and titles of the events are defined in `rules/config/channel_eid_info.txt`.

Example:

```
Channel,EventID,EventTitle
Microsoft-Windows-Sysmon/Operational,1,Process Creation.
Microsoft-Windows-Sysmon/Operational,2,File Creation Timestamp Changed. (Possible Timestomping)
Microsoft-Windows-Sysmon/Operational,3,Network Connection.
Microsoft-Windows-Sysmon/Operational,4,Sysmon Service State Changed.
```

`eid-metrics` screenshot

Total Event Records: 4,817,181				
First Timestamp: 2009-07-14 13:56:45.074 +09:00				
Last Timestamp: 2022-09-18 23:37:13.088 +09:00				
Total	%	Channel	ID	Event
1400648	29.1%	Sysmon	12	Registry Object Created/Deletion
1026221	21.3%	Sysmon	10	Process Access
976144	20.3%	Sysmon	13	Registry Value Set
609163	12.6%	Sysmon	7	Image Loaded
422040	8.8%	Sysmon	11	File Creation or Overwrite
53696	1.1%	Sysmon	26	File Deleted
35291	0.7%	MS-Win-systemdataa rchiver/diagnostic	2050	Unknown
31784	0.7%	Sysmon	2	File Creation Timestamp Changed (Possible Timestomping)

`expand-list` command

Extract `expand` placeholders from the rules folder. This is useful when creating config files to use any rule that uses the `expand` field modifier. To use `expand` rules, you just need to create a `.txt` file with the name of the `expand` field modifier under the `./config/expand/` directory, and put in all of the values you want to check inside the file.

For example, if the rule `detection` logic is:

```
detection:
  selection:
    EventID: 5145
    RelativeTargetName|contains: '\winreg'
  filter_main:
    IpAddress|expand: '%Admins_Workstations%'
  condition: selection and not filter_main
```

you would create the text file `./config/expand/Admins_Workstations.txt` and put in values like:

```
AdminWorkstation1
AdminWorkstation2
```

```
AdminWorkstation3
```

This would essentially check the same logic as:

```
- IPAddress: 'AdminWorkstation1'
- IPAddress: 'AdminWorkstation2'
- IPAddress: 'AdminWorkstation3'
```

If the config file does not exist, Hayabusa will still load the `expand` rule but ignore it.

```
Usage:  expand-list <INPUT> [OPTIONS]
```

General Options:

```
-h, --help          Show the help menu
-r, --rules <DIR/FILE> Specify rule directory (default: ./rules)
```

Display Settings:

```
-K, --no-color  Disable color output
-q, --quiet     Quiet mode: do not display the launch banner
```

expand-list command examples

- Extract out `expand` field modifiers from the default `rules` directory: `hayabusa.exe expand-list`
- Extract out `expand` field modifiers from the `sigma` directory: `hayabusa.exe eid-metrics -r ../sigma`

expand-list results

5 unique expand placeholders found:

Admins_Workstations

DC-MACHINE-NAME

Workstations

internal_domains

domain_controller_hostnames

extract-base64 command

This command will extract base64 strings from the following events, decode them and tell what kind of encoding is being used.

- Security 4688 CommandLine
- Sysmon 1 CommandLine, ParentCommandLine
- System 7045 ImagePath
- PowerShell Operational 4104
- PowerShell Operational 4103

```
Usage:  extract-base64 <INPUT> [OPTIONS]
```

Input:

```
-d, --directory <DIR>  Directory of multiple .evtx files
-f, --file <FILE>      File path to one .evtx file
-l, --live-analysis     Analyze the local C:\Windows\System32\winevt\Logs folder
```

General Options:

```
-C, --clobber          Overwrite files when saving
-h, --help             Show the help menu
-J, --JSON-input       Scan JSON formatted logs instead of .evtx (.json or
.jsonl)
-Q, --quiet-errors     Quiet errors mode: do not save error logs
```

```

-x, --recover-records          Carve evtx records from slack space (default: disabled)
-c, --rules-config <DIR>      Specify custom rule config directory (default:
./rules/config)
-t, --threads <NUMBER>        Number of threads (default: optimal number for
performance)
--target-file-ext <FILE-EXT...> Specify additional evtx file extensions (ex: evtx_data)

Filtering:
--exclude-computer <COMPUTER...> Do not scan specified computer names (ex: ComputerA)
(ex: ComputerA,ComputerB)
--include-computer <COMPUTER...> Scan only specified computer names (ex: ComputerA) (ex:
ComputerA,ComputerB)
--time-offset <OFFSET>        Scan recent events based on an offset (ex: 1y, 3M, 30d,
24h, 30m)

Output:
-o, --output <FILE>           Extract Base64 strings

Display Settings:
-K, --no-color                Disable color output
-q, --quiet                   Quiet mode: do not display the launch banner
-v, --verbose                 Output verbose information

Time Format:
--European-time               Output timestamp in European time format (ex: 22-02-2022 22:00:00.123
+02:00)
-O, --ISO-8601                Output timestamp in original ISO-8601 format (ex: 2022-02-
22T10:10:10.1234567Z) (Always UTC)
--RFC-2822                    Output timestamp in RFC 2822 format (ex: Fri, 22 Feb 2022 22:00:00
-0600)
--RFC-3339                    Output timestamp in RFC 3339 format (ex: 2022-02-22 22:00:00.123456-
06:00)
--US-military-time            Output timestamp in US military time format (ex: 02-22-2022
22:00:00.123 -06:00)
--US-time                     Output timestamp in US time format (ex: 02-22-2022 10:00:00.123 PM
-06:00)
-U, --UTC                     Output time in UTC format (default: local time)

```

extract-base64 command examples

- Scan a directory and output to the terminal: `hayabusa.exe extract-base64 -d ../hayabusa-sample-evtx`
- Scan a directory and output to a CSV file: `hayabusa.exe eid-metrics -r ../sigma -o base64-extracted.csv`

extract-base64 results

When outputting to the terminal, because space is limited, only the following fields are displayed:

- Timestamp
- Computer
- Base64 String
- Decoded String (if not binary)

When saving to a CSV file, the following fields are saved:

- Timestamp
- Computer
- Base64 String
- Decoded String (if not binary)

- Original Field
- Length
- Binary (Y/N)
- Double Encoding (when Y , it usually is malicious)
- Encoding Type
- File Type
- Event
- Record ID
- File Name

log-metrics command

You can use the `log-metrics` command to print out the following metadata inside event logs:

- Filename
- Computer names
- Number of events
- First timestamp
- Last timestamp
- Channels
- Providers

This command does not use any detection rules so will scan all events.

Usage: `log-metrics <INPUT> [OPTIONS]`

Input:

- `-d, --directory <DIR>` Directory of multiple .evtx files
- `-f, --file <FILE>` File path to one .evtx file
- `-l, --live-analysis` Analyze the local C:\Windows\System32\winevt\Logs folder

General Options:

- `-C, --clobber` Overwrite files when saving
- `-h, --help` Show the help menu
- `-J, --JSON-input` Scan JSON formatted logs instead of .evtx (.json or .jsonl)
- `-Q, --quiet-errors` Quiet errors mode: do not save error logs
- `-x, --recover-records` Carve evtx records from slack space (default: disabled)
- `-c, --rules-config <DIR>` Specify custom rule config directory (default: ./rules/config)
- `-t, --threads <NUMBER>` Number of threads (default: optimal number for performance)
- `--target-file-ext <FILE-EXT...>` Specify additional evtx file extensions (ex: evtx_data)

Filtering:

- `--exclude-computer <COMPUTER...>` Do not scan specified computer names (ex: ComputerA) (ex: ComputerA,ComputerB)
- `--include-computer <COMPUTER...>` Scan only specified computer names (ex: ComputerA) (ex: ComputerA,ComputerB)
- `--time-offset <OFFSET>` Scan recent events based on an offset (ex: 1y, 3M, 30d, 24h, 30m)

Output:

- `-b, --disable-abbreviations` Disable abbreviations
- `-M, --multiline` Output event field information in multiple rows for CSV output
- `-o, --output <FILE>` Save the Metrics in CSV format (ex: metrics.csv)
- `-S, --tab-separator` Separate event field information by tabs

Display Settings:

```
-K, --no-color  Disable color output
-q, --quiet     Quiet mode: do not display the launch banner
-v, --verbose   Output verbose information
```

Time Format:

```
--European-time  Output timestamp in European time format (ex: 22-02-2022 22:00:00.123
+02:00)
-o, --ISO-8601   Output timestamp in original ISO-8601 format (ex: 2022-02-
22T10:10:10.1234567Z) (Always UTC)
--RFC-2822       Output timestamp in RFC 2822 format (ex: Fri, 22 Feb 2022 22:00:00
-0600)
--RFC-3339       Output timestamp in RFC 3339 format (ex: 2022-02-22 22:00:00.123456-
06:00)
--US-military-time Output timestamp in US military time format (ex: 02-22-2022
22:00:00.123 -06:00)
--US-time        Output timestamp in US time format (ex: 02-22-2022 10:00:00.123 PM
-06:00)
-U, --UTC        Output time in UTC format (default: local time)
```

log-metrics command examples

- Print Event ID metrics from a single file: `hayabusa.exe log-metrics -f Security.evtx`
- Print Event ID metrics from a directory: `hayabusa.exe log-metrics -d ../logs`
- Save results to a CSV file: `hayabusa.exe log-metrics -d ../logs -o eid-metrics.csv`

log-metrics screenshot

Filename	Computers	Events	First Timestamp	Last Timestamp	Channels	Providers
many-events-security.evtx	37L4247D28-05 IE10Win7 IE9Win7 IE9Win7	13,223	2013-10-24 01:16:13.765 +09:00	2016-09-21 00:46:32.738 +09:00	Sec	MS-Win-Eventlog Sec
many-events-system.evtx	37L4247D28-05 IE10Win7 IE8Win7 IE9Win7	8,414	2009-07-14 13:56:45.074 +09:00	2016-09-21 00:45:22.402 +09:00	Sys	BROWSER BTUSB Eventlog MS-Win-App-Exp MS-Win-DHCPv6-CLI MS-Win-DNS-CLI MS-Win-Dhcp-CLI MS-Win-DistributedCOM MS-Win-FilterMgr MS-Win-GrpPolicy MS-Win-HttpEvent MS-Win-Kern-General MS-Win-Kern-Processor-Pwr MS-Win-Kern-Pwr MS-Win-Pwr-Troubleshooter MS-Win-Setup MS-Win-Svc Pack Installer MS-Win-Time-Svc MS-Win-UserPrp MS-Win-WMPNSS-Svc MS-Win-Winlogon NetBT PNPMEM RDS-RCM SCH Tcpip USER32 WinUpdate netvsc storflt wmi volmgr

logon-summary command

You can use the `logon-summary` command to output logon information summary (logon usernames and successful and failed logon count). You can display the logon information for one evtx file with `-f` or multiple evtx files with the `-d` option.

Successful logons are taken from the following events:

- Security 4624 (Successful Logon)
- RDS-LSM 21 (Remote Desktop Service Local Session Manager Logon)
- RDS-GTW 302 (Remote Desktop Service Gateway Logon)

Failed logons are taken from Security 4625 events.

Usage: logon-summary <INPUT> [OPTIONS]

Input:

-d, --directory <DIR> Directory of multiple .evtx files
-f, --file <FILE> File path to one .evtx file
-l, --live-analysis Analyze the local C:\Windows\System32\winevt\Logs folder

General Options:

-C, --clobber Overwrite files when saving
-h, --help Show the help menu
-J, --JSON-input Scan JSON formatted logs instead of .evtx (.json or .jsonl)
-Q, --quiet-errors Quiet errors mode: do not save error logs
-x, --recover-records Carve evtx records from slack space (default: disabled)
-c, --rules-config <DIR> Specify custom rule config directory (default: ./rules/config)
-t, --threads <NUMBER> Number of threads (default: optimal number for performance)
--target-file-ext <FILE-EXT...> Specify additional evtx file extensions (ex: evtx_data)

Filtering:

--exclude-computer <COMPUTER...> Do not scan specified computer names (ex: ComputerA) (ex: ComputerA,ComputerB)
--include-computer <COMPUTER...> Scan only specified computer names (ex: ComputerA) (ex: ComputerA,ComputerB)
--time-offset <OFFSET> Scan recent events based on an offset (ex: 1y, 3M, 30d, 24h, 30m)
--timeline-end <DATE> End time of the event logs to load (ex: "2022-02-22 23:59:59 +09:00")
--timeline-start <DATE> Start time of the event logs to load (ex: "2020-02-22 00:00:00 +09:00")

Output:

-o, --output <FILENAME-PREFIX> Save the logon summary to two CSV files (ex: -o logon-summary)

Display Settings:

-K, --no-color Disable color output
-q, --quiet Quiet mode: do not display the launch banner
-v, --verbose Output verbose information

Time Format:

--European-time Output timestamp in European time format (ex: 22-02-2022 22:00:00.123 +02:00)
-O, --ISO-8601 Output timestamp in original ISO-8601 format (ex: 2022-02-22T10:10:10.1234567Z) (Always UTC)
--RFC-2822 Output timestamp in RFC 2822 format (ex: Fri, 22 Feb 2022 22:00:00 -0600)
--RFC-3339 Output timestamp in RFC 3339 format (ex: 2022-02-22 22:00:00.123456-06:00)
--US-military-time Output timestamp in US military time format (ex: 02-22-2022 22:00:00.123 -06:00)
--US-time Output timestamp in US time format (ex: 02-22-2022 10:00:00.123 PM -06:00)
-U, --UTC Output time in UTC format (default: local time)

logon-summary command examples

- Print logon summary: `hayabusa.exe logon-summary -f Security.evtx`
- Save logon summary results: `hayabusa.exe logon-summary -d ../logs -o logon-summary.csv`

Logon-summary screenshots

Total Event Records: 4,817,181

First Timestamp: 2009-07-14 13:56:45.074 +09:00
Last Timestamp: 2022-09-18 23:37:13.088 +09:00

Logon Summary:

Successful Logons:

Successful	Target Account	Target Computer	Logon Type	Source Computer	Source IP Address
221	Système	DESKTOP-6D0DBMB	5 - Service	-	-
212	SYSTEM	DESKTOP-A8CALR3	5 - Service	-	-
63	SYSTEM	WIN-FPV0DSIC906	5 - Service	-	-
51	SYSTEM	evtx-PC	5 - Service	-	-
34	Système	WIN-KP04DDU11AB	5 - Service	-	-
30	SYSTEM	WIN-TKC15D7KHUR	5 - Service	-	-
16	user	DESKTOP-A8CALR3	2 - Interactive	DESKTOP-A8CALR3	127.0.0.1
12	SYSTEM	Agamemnon	5 - Service	-	-

Failed Logons:

Failed	Target Account	Target Computer	Logon Type	Source Computer	Source IP Address
2	testme	DESKTOP-6D0DBMB	2 - Interactive	WIN-KP04DDU11AB	-
1	user	DESKTOP-A8CALR3	2 - Interactive	DESKTOP-A8CALR3	127.0.0.1

pivot-keywords-list command

You can use the `pivot-keywords-list` command to create a list of unique pivot keywords to quickly identify abnormal users, hostnames, processes, etc... as well as correlate events.

Important: by default, hayabusa will return results from all events (informational and higher) so we highly recommend combining the `pivot-keywords-list` command with the `-m, --min-level` option. For example, start off with only creating keywords from `critical` alerts with `-m critical` and then continue with `-m high`, `-m medium`, etc... There will most likely be common keywords in your results that will match on many normal events, so after manually checking the results and creating a list of unique keywords in a single file, you can then create a narrowed down timeline of suspicious activity with a command like `grep -f keywords.txt timeline.csv`.

Usage: `pivot-keywords-list <INPUT> [OPTIONS]`

Input:

- d, --directory <DIR> Directory of multiple .evtx files
- f, --file <FILE> File path to one .evtx file
- l, --live-analysis Analyze the local C:\Windows\System32\winevt\Logs folder

General Options:

```

-C, --clobber           Overwrite files when saving
-h, --help             Show the help menu
-J, --JSON-input       Scan JSON formatted logs instead of .evtx (.json or
.jsonl)
-w, --no-wizard        Do not ask questions. Scan for all events and alerts
-Q, --quiet-errors     Quiet errors mode: do not save error logs
-x, --recover-records  Carve evtx records from slack space (default: disabled)
-c, --rules-config <DIR> Specify custom rule config directory (default:
./rules/config)
-t, --threads <NUMBER> Number of threads (default: optimal number for
performance)
--target-file-ext <FILE-EXT...> Specify additional evtx file extensions (ex: evtx_data)

Filtering:
-E, --EID-filter       Scan only common EIDs for faster speed
(./rules/config/target_event_IDs.txt)
-D, --enable-deprecated-rules Enable rules with a status of deprecated
-n, --enable-noisy-rules  Enable rules set to noisy
(./rules/config/noisy_rules.txt)
-u, --enable-unsupported-rules Enable rules with a status of unsupported
-e, --exact-level <LEVEL> Only load rules with a specific level (informational,
low, medium, high, critical)
--exclude-computer <COMPUTER...> Do not scan specified computer names (ex: ComputerA)
(ex: ComputerA,ComputerB)
--exclude-eid <EID...> Do not scan specific EIDs for faster speed (ex: 1) (ex:
1,4688)
--exclude-status <STATUS...> Do not load rules according to status (ex:
experimental) (ex: stable,test)
--exclude-tag <TAG...> Do not load rules with specific tags (ex: sysmon)
--include-computer <COMPUTER...> Scan only specified computer names (ex: ComputerA) (ex:
ComputerA,ComputerB)
--include-eid <EID...> Scan only specified EIDs for faster speed (ex: 1) (ex:
1,4688)
--include-status <STATUS...> Only load rules with specific status (ex: experimental)
(ex: stable,test)
--include-tag <TAG...> Only load rules with specific tags (ex:
attack.execution,attack.discovery)
-m, --min-level <LEVEL> Minimum level for rules to load (default:
informational)
--time-offset <OFFSET> Scan recent events based on an offset (ex: 1y, 3M, 30d,
24h, 30m)
--timeline-end <DATE> End time of the event logs to load (ex: "2022-02-22
23:59:59 +09:00")
--timeline-start <DATE> Start time of the event logs to load (ex: "2020-02-22
00:00:00 +09:00")

Output:
-o, --output <FILENAME-PREFIX> Save pivot words to separate files (ex: PivotKeywords)

Display Settings:
-K, --no-color Disable color output
-q, --quiet Quiet mode: do not display the launch banner
-v, --verbose Output verbose information

```

pivot-keywords-list command examples

- Output pivot keywords to screen: `hayabusa.exe pivot-keywords-list -d ../logs -m critical`

- Create a list of pivot keywords from critical alerts and save the results. (Results will be saved to `keywords-IpAddresses.txt` , `keywords-Users.txt` , etc...):

```
hayabusa.exe pivot-keywords-list -d ../logs -m critical -o keywords`
```

pivot-keywords-list config file

You can customize what keywords you want to search for by editing `./rules/config/pivot_keywords.txt` . [This page](#) is the default setting.

The format is `KeywordName.FieldName` . For example, when creating the list of `Users` , hayabusa will list up all the values in the `SubjectUserName` , `TargetUserName` and `User` fields.

search command

The `search` command will let you keyword search on all events. (Not just Hayabusa detection results.) This is useful to determine if there is any evidence in events that are not detected by Hayabusa.

Usage: `hayabusa.exe search <INPUT> <--keywords "<KEYWORDS>" OR --regex "<REGEX>"> [OPTIONS]`

Display Settings:

```
-K, --no-color  Disable color output
-q, --quiet     Quiet mode: do not display the launch banner
-v, --verbose   Output verbose information
```

General Options:

```
-C, --clobber           Overwrite files when saving
-h, --help             Show the help menu
-Q, --quiet-errors     Quiet errors mode: do not save error logs
-x, --recover-records  Carve evtx records from slack space (default: disabled)
-c, --rules-config <DIR> Specify custom rule config directory (default:
./rules/config)
-t, --threads <NUMBER> Number of threads (default: optimal number for
performance)
--target-file-ext <FILE-EXT...> Specify additional evtx file extensions (ex: evtx_data)
-s, --sort             Sort results before saving the file (warning: this uses
much more memory!)
```

Input:

```
-d, --directory <DIR>  Directory of multiple .evtx files
-f, --file <FILE>      File path to one .evtx file
-l, --live-analysis    Analyze the local C:\Windows\System32\winevt\Logs folder
```

Filtering:

```
-a, --and-logic        Search keywords with AND logic (default: OR)
-F, --filter <FILTER...> Filter by specific field(s)
-i, --ignore-case      Case-insensitive keyword search
-k, --keyword <KEYWORD...> Search by keyword(s)
-r, --regex <REGEX>   Search by regular expression
--time-offset <OFFSET> Scan recent events based on an offset (ex: 1y, 3M, 30d, 24h,
30m)
--timeline-end <DATE>  End time of the event logs to load (ex: "2022-02-22 23:59:59
+09:00")
--timeline-start <DATE> Start time of the event logs to load (ex: "2020-02-22 00:00:00
+09:00")
```

Output:

```
-b, --disable-abbreviations  Disable abbreviations
```

<code>-J, --JSON-output</code>	Save the search results in JSON format (ex: <code>-J -o results.json</code>)
<code>-L, --JSONL-output</code>	Save the search results in JSONL format (ex: <code>-L -o results.jsonl</code>)
<code>-M, --multiline</code>	Output event field information in multiple rows for CSV output
<code>-o, --output <FILE></code>	Save the search results in CSV format (ex: <code>search.csv</code>)
<code>-S, --tab-separator</code>	Separate event field information by tabs

Time Format:

<code>--European-time</code>	Output timestamp in European time format (ex: <code>22-02-2022 22:00:00.123+02:00</code>)
<code>-0, --ISO-8601</code>	Output timestamp in original ISO-8601 format (ex: <code>2022-02-22T10:10:10.1234567Z</code>) (Always UTC)
<code>--RFC-2822</code>	Output timestamp in RFC 2822 format (ex: <code>Fri, 22 Feb 2022 22:00:00-0600</code>)
<code>--RFC-3339</code>	Output timestamp in RFC 3339 format (ex: <code>2022-02-22 22:00:00.123456-06:00</code>)
<code>--US-military-time</code>	Output timestamp in US military time format (ex: <code>02-22-2022 22:00:00.123 -06:00</code>)
<code>--US-time</code>	Output timestamp in US time format (ex: <code>02-22-2022 10:00:00.123 PM -06:00</code>)
<code>-U, --UTC</code>	Output time in UTC format (default: local time)

search command examples

- Search the `../hayabusa-sample-evtx` directory for the keyword `mimikatz` :

```
hayabusa.exe search -d ../hayabusa-sample-evtx -k "mimikatz"
```

Note: The keyword will match if `mimikatz` is found anywhere in the data. It is not an exact match.

- Search the `../hayabusa-sample-evtx` directory for the keywords `mimikatz` or `kali` :

```
hayabusa.exe search -d ../hayabusa-sample-evtx -k "mimikatz" -k "kali"
```

- Search the `../hayabusa-sample-evtx` directory for the keyword `mimikatz` and ignore case:

```
hayabusa.exe search -d ../hayabusa-sample-evtx -k "mimikatz" -i
```

- Search the `../hayabusa-sample-evtx` directory for IP addresses using regular expressions:

```
hayabusa.exe search -d ../hayabusa-sample-evtx -r "(?:[0-9]{1,3}\.){3}[0-9]{1,3}"
```

- Search the `../hayabusa-sample-evtx` directory and show all events where the `WorkstationName` field is `kali` :

```
hayabusa.exe search -d ../hayabusa-sample-evtx -r ".*" -F WorkstationName:"kali"
```

Note: `.*` is the regular expression to match on every event.

search command config files

`./rules/config/channel_abbreviations.txt` : Mappings of channel names and their abbreviations.

Config Commands

config-critical-systems command

This command will automatically try to find critical systems like domain controllers and file servers and add them to the `./config/critical_systems.txt` config file so that all of the alerts will be increased by one level. It will search for Security 4768 (Kerberos TGT requested) events to determine if it is a domain controller. It will search for Security 5145 (Network Share File Access) events to determine if it is a file server. Any hostnames added to the `critical_systems.txt` file will have all alerts above low increased by one level with a maximum of `emergency` level.

```
Usage: hayabusa.exe config-critical-systems <INPUT> [OPTIONS]
```

Input:

```
-d, --directory <DIR>  Directory of multiple .evtx files
-f, --file <FILE>      File path to one .evtx file
```

Display Settings:

```
-K, --no-color  Disable color output
-q, --quiet     Quiet mode: do not display the launch banner
```

General Options:

```
-h, --help  Show the help menu
```

`config-critical-systems` command examples

- Search the `../hayabusa-sample-evtx` directory for domain controllers and file servers:

```
hayabusa.exe config-critical-systems -d ../hayabusa-sample-evtx"
```

DFIR Timeline Commands

Scan Wizard

The `csv-timeline` and `json-timeline` commands now have a scan wizard enabled by default. This is intended to help users easily choose which detection rules they want to enable according to their needs and preferences. The sets of detections rules to load are based off of the official lists in the Sigma project. Details are explained in [this blog post](#). You can easily turn off the wizard and use Hayabusa in its traditional way by adding the `-w, --no-wizard` option.

Core Rules

The `core` rule set enables rules that have a status of `test` or `stable` and a level of `high` or `critical`. These are high quality rules of high confidence and relevance and should not produce many false positives. The rule status is `test` or `stable` which means no false positives were reported for over 6 months. Rules will match on attacker techniques, generic suspicious activity, or malicious behavior. It is the same as using the `--exclude-status deprecated,unsupported,experimental --min-level high` options.

Core+ Rules

The `core+` rule set enables rules that have a status of `test` or `stable` and a level of `medium` or higher. `medium` rules most often need additional tuning as certain applications, legitimate user behavior or scripts of an organization might be matched. It is the same as using the `--exclude-status deprecated,unsupported,experimental --min-level medium` options.

Core++ Rules

The `core++` rule set enables rules that have a status of `experimental`, `test` or `stable` and a level of `medium` or higher. These rules are bleeding edge. They are validated against the baseline evtx files available at the SigmaHQ project and reviewed by multiple detection engineers. Other than that they are pretty much untested at first. Use these if you want to be able to detect threats as early as possible at the cost of managing a higher threshold of false positives. It is the same as using the `--exclude-status deprecated,unsupported --min-level medium` options.

Emerging Threats (ET) Add-On Rules

The `Emerging Threats (ET)` rule set enables rules that have a tag of `detection.emerging_threats`. These rules target specific threats and are especially useful for current threats where not much information is available yet. These rules should not have many false positives but will decrease in relevance over time. When these rules are not enabled, it is the same as using the `--exclude-tag detection.emerging_threats` option. When running Hayabusa traditionally without the wizard, these rules will be included by default.

Threat Hunting (TH) Add-On Rules

The `Threat Hunting (TH)` rule set enables rules that have a tag of `detection.threat_hunting`. These rules may detect unknown malicious activity, however, will typically have more false positives. When these rules are not enabled, it is the same as using the `--exclude-tag detection.threat_hunting` option. When running Hayabusa traditionally without the wizard, these rules will be included by default.

Channel-based event log and rules filtering

As of Hayabusa v2.16.0, we enable a Channel-based filter when loading `.evtx` files and `.yaml` rules. The purpose is to make scanning as efficient as possible by only loading what is necessary. While it is possible for there to be multiple providers in a single event log, it is not common to have multiple channels inside a single evtx file. (The only time we have seen this is when someone has artificially merged two different evtx files together for the [sample-evtx](#) project.) We can use this to our advantage by first checking the `Channel` field in the first record of every `.evtx` file specified to be scanned. We also check which `.yaml` rules use what channels specified in the `Channel` field of the rule. With these two lists, we only load rules that use channels that are actually present inside the `.evtx` files.

So for example, if a user wants to scan `Security.evtx`, only rules that specify `Channel: Security` will be used. There is no point in loading other detection rules, for example rules that only look for events in the `Application` log, etc... Note that channel fields (Ex: `Channel: Security`) are not **explicitly** defined inside original Sigma rules. For Sigma rules, channel and event IDs fields are **implicitly** defined with `service` and `category` fields under `logsource`. (Ex: `service: security`) When curating Sigma rules in the [hayabusa-rules](#) repository, we de-abstract the `logsource` field and explicitly define the channel and event ID fields. We explain how and why we do this in-depth [here](#).

Currently, there are only two detection rules that do not have `Channel` defined and are intended to scan all `.evtx` files are the following:

- [Possible Hidden Shellcode](#)
- [Mimikatz Use](#)

If you want to use these two rules and scan all rules against loaded `.evtx` files then you will need to add the `-A, --enable-all-rules` option in the `csv-timeline` and `json-timeline` commands. In our benchmarks, the rules filtering usually gives a 20% to 10x speed improvement depending on what files are being scanned and of course uses less memory.

Channel filtering is also used when loading `.evtx` files. For example, if you specify a rule that looks for events with a channel of `Security`, then there is no point in loading `.evtx` files that are not from the `Security` log. In our benchmarks, this gives a speed benefit of around 10% with normal scans and up to 60%+ performance increase when scanning with a single rule. If you are sure that multiple channels are being used inside a single `.evtx` file, for example someone used a tool to merge multiple `.evtx` files together, then you disable this filtering with the `-a, --scan-all-evtx-files` option in `csv-timeline` and `json-timeline` commands.

Note: Channel filtering only works with `.evtx` files and you will receive an error if you try to load event logs from a JSON file with `-J, --json-input` and also specify `-A` or `-a`.

csv-timeline command

The `csv-timeline` command will create a forensics timeline of events in CSV format.

```
Usage: csv-timeline <INPUT> [OPTIONS]
```

Input:

```
-d, --directory <DIR> Directory of multiple .evtx files
```

```

-f, --file <FILE>      File path to one .evtx file
-l, --live-analysis     Analyze the local C:\Windows\System32\winevt\Logs folder

General Options:
-C, --clobber           Overwrite files when saving
-h, --help              Show the help menu
-J, --JSON-input        Scan JSON formatted logs instead of .evtx (.json or
.jsonl)
-w, --no-wizard         Do not ask questions. Scan for all events and alerts
-Q, --quiet-errors      Quiet errors mode: do not save error logs
-x, --recover-records   Carve evtx records from slack space (default: disabled)
-r, --rules <DIR/FILE> Specify a custom rule directory or file (default:
./rules)
-c, --rules-config <DIR> Specify custom rule config directory (default:
./rules/config)
-s, --sort              Sort events before saving the file. (warning: this uses
much more memory!)
-t, --threads <NUMBER> Number of threads (default: optimal number for
performance)
--target-file-ext <FILE-EXT...> Specify additional evtx file extensions (ex: evtx_data)

Filtering:
-E, --EID-filter        Scan only common EIDs for faster speed
(./rules/config/target_event_IDs.txt)
-A, --enable-all-rules Enable all rules regardless of loaded evtx files
(disable channel filter for rules)
-D, --enable-deprecated-rules Enable rules with a status of deprecated
-n, --enable-noisy-rules Enable rules set to noisy
(./rules/config/noisy_rules.txt)
-u, --enable-unsupported-rules Enable rules with a status of unsupported
-e, --exact-level <LEVEL> Only load rules with a specific level (informational,
low, medium, high, critical)
--exclude-category <CATEGORY...> Do not load rules with specified logsource categories
(ex: process_creation,pipe_created)
--exclude-computer <COMPUTER...> Do not scan specified computer names (ex: ComputerA)
(ex: ComputerA,ComputerB)
--exclude-eid <EID...> Do not scan specific EIDs for faster speed (ex: 1) (ex:
1,4688)
--exclude-status <STATUS...> Do not load rules according to status (ex:
experimental) (ex: stable,test)
--exclude-tag <TAG...> Do not load rules with specific tags (ex: sysmon)
--include-category <CATEGORY...> Only load rules with specified logsource categories
(ex: process_creation,pipe_created)
--include-computer <COMPUTER...> Scan only specified computer names (ex: ComputerA) (ex:
ComputerA,ComputerB)
--include-eid <EID...> Scan only specified EIDs for faster speed (ex: 1) (ex:
1,4688)
--include-status <STATUS...> Only load rules with specific status (ex: experimental)
(ex: stable,test)
--include-tag <TAG...> Only load rules with specific tags (ex:
attack.execution,attack.discovery)
-m, --min-level <LEVEL> Minimum level for rules to load (default:
informational)
-P, --proven-rules      Scan with only proven rules for faster speed
(./rules/config/proven_rules.txt)
-a, --scan-all-evtx-files Scan all evtx files regardless of loaded rules (disable
channel filter for evtx files)
--time-offset <OFFSET> Scan recent events based on an offset (ex: 1y, 3M, 30d,

```

24h, 30m)

--timeline-end <DATE> End time of the event logs to load (ex: "2022-02-22 23:59:59 +09:00")

--timeline-start <DATE> Start time of the event logs to load (ex: "2020-02-22 00:00:00 +09:00")

Output:

-b, --disable-abbreviations Disable abbreviations

-G, --GeoIP <MAXMIND-DB-DIR> Add GeoIP (ASN, city, country) info to IP addresses

-H, --HTML-report <FILE> Save Results Summary details to an HTML report (ex: results.html)

-M, --multiline Output event field information in multiple rows

-F, --no-field-data-mapping Disable field data mapping

--no-pwsh-field-extraction Disable field extraction of PowerShell classic logs

-o, --output <FILE> Save the timeline in CSV format (ex: results.csv)

-p, --profile <PROFILE> Specify output profile

-R, --remove-duplicate-data Duplicate field data will be replaced with "DUP"

-X, --remove-duplicate-detections Remove duplicate detections (default: disabled)

-S, --tab-separator Separate event field information by tabs

Display Settings:

-K, --no-color Disable color output

-N, --no-summary Do not display Results Summary for faster speed

-q, --quiet Quiet mode: do not display the launch banner

-v, --verbose Output verbose information

-T, --visualize-timeline Output event frequency timeline (terminal needs to support unicode)

Time Format:

--European-time Output timestamp in European time format (ex: 22-02-2022 22:00:00.123 +02:00)

-O, --ISO-8601 Output timestamp in original ISO-8601 format (ex: 2022-02-22T10:10:10.1234567Z) (Always UTC)

--RFC-2822 Output timestamp in RFC 2822 format (ex: Fri, 22 Feb 2022 22:00:00 -0600)

--RFC-3339 Output timestamp in RFC 3339 format (ex: 2022-02-22 22:00:00.123456-06:00)

--US-military-time Output timestamp in US military time format (ex: 02-22-2022 22:00:00.123 -06:00)

--US-time Output timestamp in US time format (ex: 02-22-2022 10:00:00.123 PM -06:00)

-U, --UTC Output time in UTC format (default: local time)

csv-timeline command examples

- Run hayabusa against one Windows event log file with default standard profile:

```
hayabusa.exe csv-timeline -f eventlog.evtx
```

- Run hayabusa against the sample-evtx directory with multiple Windows event log files with the verbose profile:

```
hayabusa.exe csv-timeline -d .\hayabusa-sample-evtx -p verbose
```

- Export to a single CSV file for further analysis with LibreOffice, Timeline Explorer, Elastic Stack, etc... and include all field information (Warning: your file output size will become much larger with the super-verbose profile!):

```
hayabusa.exe csv-timeline -d .\hayabusa-sample-evtx -o results.csv -p super-verbose
```

- Enable the EID (Event ID) filter:

Note: Enabling the EID filter will speed up the analysis by about 10-15% in our tests but there is a possibility of missing alerts.

```
hayabusa.exe csv-timeline -E -d .\hayabusa-sample-evtx -o results.csv
```

- Only run hayabusa rules (the default is to run all the rules in `-r .\rules`):

```
hayabusa.exe csv-timeline -d .\hayabusa-sample-evtx -r .\rules\hayabusa -o results.csv -w
```

- Only run hayabusa rules for logs that are enabled by default on Windows:

```
hayabusa.exe csv-timeline -d .\hayabusa-sample-evtx -r .\rules\hayabusa\builtin -o results.csv -w
```

- Only run hayabusa rules for sysmon logs:

```
hayabusa.exe csv-timeline -d .\hayabusa-sample-evtx -r .\rules\hayabusa\sysmon -o results.csv -w
```

- Only run sigma rules:

```
hayabusa.exe csv-timeline -d .\hayabusa-sample-evtx -r .\rules\sigma -o results.csv -w
```

- Enable deprecated rules (those with `status` marked as `deprecated`) and noisy rules (those whose rule ID is listed in `.\rules\config\noisy_rules.txt`):

Note: Recently, deprecated rules are now located in a separate directory in the sigma repository so are not included by default anymore in Hayabusa. Therefore, you probably have no need to enable deprecated rules.

```
hayabusa.exe csv-timeline -d .\hayabusa-sample-evtx --enable-noisy-rules --enable-deprecated-rules -o results.csv -w
```

- Only run rules to analyze logons and output in the UTC timezone:

```
hayabusa.exe csv-timeline -d .\hayabusa-sample-evtx -r .\rules\hayabusa\builtin\Security\LogonLogoff\Logon -U -o results.csv -w
```

- Run on a live Windows machine (requires Administrator privileges) and only detect alerts (potentially malicious behavior):

```
hayabusa.exe csv-timeline -l -m low
```

- Print verbose information (useful for determining which files take long to process, parsing errors, etc...):

```
hayabusa.exe csv-timeline -d .\hayabusa-sample-evtx -v
```

- Verbose output example:

Loading rules:

```
Loaded rule: rules/sigma/builtin/deprecated/proc_creation_win_susp_run_folder.yml
Loaded rule: rules/sigma/builtin/deprecated/proc_creation_win_execution_mssql_xp_cmdshell_stored_procedure.yml
Loaded rule: rules/sigma/builtin/deprecated/proc_creation_win_susp_squirrel_lolbin.yml
Loaded rule: rules/sigma/builtin/win_alert_mimikatz_keywords.yml
```

Errors during the scan:

```
[ERROR] Failed to parse event file.
EventFile: ../logs/Microsoft-Rdms-UI%40operational.evtx
Error: Failed to parse record number 58471

[ERROR] Failed to parse event file.
EventFile: ../logs/Microsoft-Rdms-UI%40operational.evtx
Error: Failed to parse record number 58470

[ERROR] Failed to parse event file.
EventFile: ../logs/Microsoft-Windows-AppxPackaging%40operational.evtx
Error: An error occurred while trying to serialize binary xml to output.
```

- Output to a CSV format compatible to import into [Timesketch](#):

```
hayabusa.exe csv-timeline -d ../hayabusa-sample-evtx --RFC-3339 -o timesketch-import.csv -p timesketch -U
```

- Quiet error mode: By default, hayabusa will save error messages to error log files. If you do not want to save error messages, please add `-Q` .

Advanced - GeoIP Log Enrichment

You can add GeoIP (ASN organization, city and country) information to SrcIP (source IP) fields and TgtIP (target IP) fields with the free GeoLite2 geolocation data.

Steps:

1. First sign up for a MaxMind account [here](#).
 2. Download the three `.mmdb` files from the [download page](#) and save them to a directory. The filenames should be called `GeoLite2-ASN.mmdb` , `GeoLite2-City.mmdb` and `GeoLite2-Country.mmdb` .
 3. When running the `csv-timeline` or `json-timeline` commands, add the `-G` option followed by the directory with the MaxMind databases.
- When `csv-timeline` is used, the following 6 columns will be additionally outputted: `SrcASN` , `SrcCity` , `SrcCountry` , `TgtASN` , `TgtCity` , `TgtCountry` .
 - When `json-timeline` is used, the same `SrcASN` , `SrcCity` , `SrcCountry` , `TgtASN` , `TgtCity` , `TgtCountry` fields will be added to the `Details` object, but only if they contain information.
 - When `SrcIP` or `TgtIP` is localhost (`127.0.0.1` , `::1` , etc...), `SrcASN` or `TgtASN` will be outputted as `Local` .
 - When `SrcIP` or `TgtIP` is a private IP address (`10.0.0.0/8` , `fe80::/10` , etc...), `SrcASN` or `TgtASN` will be outputted as `Private` .

GeoIP config file

The field names that contain source and target IP addresses that get looked up in the GeoIP databases are defined in `rules/config/geoip_field_mapping.yaml` . You can add to this list if necessary. There is also a filter section in this file that determines what events to extract IP address information from.

Automatic updates of GeoIP databases

MaxMind GeoIP databases are updated every 2 weeks. You can install the MaxMind `geoipupdate` tool [here](#) in order to automatically update these databases.

Steps on macOS:

1. `brew install geoipupdate`

2. Edit `/usr/local/etc/GeoIP.conf` or `/opt/homebrew/etc/GeoIP.conf` : Put in your `AccountID` and `LicenseKey` you create after logging into the MaxMind website. Make sure the `EditionIDs` line says `EditionIDs GeoLite2-ASN GeoLite2-City GeoLite2-Country` .
3. Run `geoipupdate` .
4. Add `-G /usr/local/var/GeoIP` or `-G /opt/homebrew/var/GeoIP` when you want to add GeoIP information.

Steps on Windows:

1. Download the latest Windows binary (Ex: `geoipupdate_4.10.0_windows_amd64.zip`) from the [Releases](#) page.
2. Edit `\ProgramData\MaxMind\GeoIPUpdate\GeoIP.conf` : Put in your `AccountID` and `LicenseKey` you create after logging into the MaxMind website. Make sure the `EditionIDs` line says `EditionIDs GeoLite2-ASN GeoLite2-City GeoLite2-Country` .
3. Run the `geoipupdate` executable.

csv-timeline command config files

`./rules/config/channel_abbreviations.txt` : Mappings of channel names and their abbreviations.

`./rules/config/default_details.txt` : The configuration file for what default field information (`%Details%` field) should be outputted if no `details:` line is specified in a rule. This is based on provider name and event IDs.

`./rules/config/eventkey_alias.txt` : This file has the mappings of short name aliases for fields and their original longer field names.

Example:

```
InstanceID,Event.UserData.UMDFHostDeviceArrivalBegin.InstanceId
IntegrityLevel,Event.EventData.IntegrityLevel
IpAddress,Event.EventData.IpAddress
```

If a field is not defined here, Hayabusa will automatically check under `Event.EventData` for the field.

`./rules/config/exclude_rules.txt` : This file has a list of rule IDs that will be excluded from use. Usually this is because one rule has replaced another or the rule cannot be used in the first place. Like firewalls and IDSes, any signature-based tool will require some tuning to fit your environment so you may need to permanently or temporarily exclude certain rules. You can add a rule ID (Example: `4fe151c2-ecf9-4fae-95ae-b88ec9c2fca6`) to

`./rules/config/exclude_rules.txt` in order to ignore any rule that you do not need or cannot be used.

`./rules/config/noisy_rules.txt` : This file a list of rule IDs that are disabled by default but can be enabled by enabling noisy rules with the `-n, --enable-noisy-rules` option. These rules are usually noisy by nature or due to false positives.

`./rules/config/target_event_IDs.txt` : Only the event IDs specified in this file will be scanned if the EID filter is enabled. By default, Hayabusa will scan all events, but if you want to improve performance, please use the `-E, --EID-filter` option. This usually results in a 10~25% speed improvement.

json-timeline command

The `json-timeline` command will create a forensics timeline of events in JSON or JSONL format. Outputting to JSONL will be faster and smaller file size than JSON so is good if you are going to just import the results into another tool like Elastic Stack. JSON is better if you are going to manually analyze the results with a text editor. CSV output is good for importing smaller timelines (usually less than 2GB) into tools like LibreOffice or Timeline Explorer. JSON is best for more detailed analysis of data (including large results files) with tools like `jq` as the `Details` fields are separated for easier analysis. (In the CSV output, all of the event log fields are in one big `Details` column making sorting of data, etc... more difficult.)

```
Usage: json-timeline <INPUT> [OPTIONS]
```

Input:

-d, --directory <DIR> Directory of multiple .evtx files
-f, --file <FILE> File path to one .evtx file
-l, --live-analysis Analyze the local C:\Windows\System32\winevt\Logs folder

General Options:

-C, --clobber Overwrite files when saving
-h, --help Show the help menu
-J, --JSON-input Scan JSON formatted logs instead of .evtx (.json or .jsonl)
-w, --no-wizard Do not ask questions. Scan for all events and alerts
-Q, --quiet-errors Quiet errors mode: do not save error logs
-x, --recover-records Carve evtx records from slack space (default: disabled)
-r, --rules <DIR/FILE> Specify a custom rule directory or file (default: ./rules)
-c, --rules-config <DIR> Specify custom rule config directory (default: ./rules/config)
-s, --sort Sort events before saving the file. (warning: this uses much more memory!)
-t, --threads <NUMBER> Number of threads (default: optimal number for performance)
--target-file-ext <FILE-EXT...> Specify additional evtx file extensions (ex: evtx_data)

Filtering:

-E, --EID-filter Scan only common EIDs for faster speed
(./rules/config/target_event_IDs.txt)
-A, --enable-all-rules Enable all rules regardless of loaded evtx files
(disable channel filter for rules)
-D, --enable-deprecated-rules Enable rules with a status of deprecated
-n, --enable-noisy-rules Enable rules set to noisy
(./rules/config/noisy_rules.txt)
-u, --enable-unsupported-rules Enable rules with a status of unsupported
-e, --exact-level <LEVEL> Only load rules with a specific level (informational, low, medium, high, critical)
--exclude-category <CATEGORY...> Do not load rules with specified logsource categories
(ex: process_creation,pipe_created)
--exclude-computer <COMPUTER...> Do not scan specified computer names (ex: ComputerA)
(ex: ComputerA,ComputerB)
--exclude-eid <EID...> Do not scan specific EIDs for faster speed (ex: 1) (ex: 1,4688)
--exclude-status <STATUS...> Do not load rules according to status (ex: experimental) (ex: stable,test)
--exclude-tag <TAG...> Do not load rules with specific tags (ex: sysmon)
--include-category <CATEGORY...> Only load rules with specified logsource categories
(ex: process_creation,pipe_created)
--include-computer <COMPUTER...> Scan only specified computer names (ex: ComputerA) (ex: ComputerA,ComputerB)
--include-eid <EID...> Scan only specified EIDs for faster speed (ex: 1) (ex: 1,4688)
--include-status <STATUS...> Only load rules with specific status (ex: experimental) (ex: stable,test)
--include-tag <TAG...> Only load rules with specific tags (ex: attack.execution,attack.discovery)
-m, --min-level <LEVEL> Minimum level for rules to load (default: informational)
-P, --proven-rules Scan with only proven rules for faster speed
(./rules/config/proven_rules.txt)
-a, --scan-all-evtx-files Scan all evtx files regardless of loaded rules (disable

```

channel filter for evtx files)
    --time-offset <OFFSET>          Scan recent events based on an offset (ex: 1y, 3M, 30d,
24h, 30m)
    --timeline-end <DATE>            End time of the event logs to load (ex: "2022-02-22
23:59:59 +09:00")
    --timeline-start <DATE>          Start time of the event logs to load (ex: "2020-02-22
00:00:00 +09:00")

Output:
    -b, --disable-abbreviations      Disable abbreviations
    -G, --GeoIP <MAXMIND-DB-DIR>     Add GeoIP (ASN, city, country) info to IP addresses
    -H, --HTML-report <FILE>         Save Results Summary details to an HTML report (ex:
results.html)
    -L, --JSONL-output               Save the timeline in JSONL format (ex: -L -o
results.jsonl)
    -F, --no-field-data-mapping      Disable field data mapping
    --no-pwsh-field-extraction       Disable field extraction of PowerShell classic logs
    -o, --output <FILE>              Save the timeline in JSON format (ex: results.json)
    -p, --profile <PROFILE>          Specify output profile
    -R, --remove-duplicate-data       Duplicate field data will be replaced with "DUP"
    -X, --remove-duplicate-detections Remove duplicate detections (default: disabled)

Display Settings:
    -K, --no-color                   Disable color output
    -N, --no-summary                 Do not display Results Summary for faster speed
    -q, --quiet                       Quiet mode: do not display the launch banner
    -v, --verbose                     Output verbose information
    -T, --visualize-timeline          Output event frequency timeline (terminal needs to support unicode)

Time Format:
    --European-time                  Output timestamp in European time format (ex: 22-02-2022 22:00:00.123
+02:00)
    -O, --ISO-8601                  Output timestamp in original ISO-8601 format (ex: 2022-02-
22T10:10:10.1234567Z) (Always UTC)
    --RFC-2822                       Output timestamp in RFC 2822 format (ex: Fri, 22 Feb 2022 22:00:00
-0600)
    --RFC-3339                       Output timestamp in RFC 3339 format (ex: 2022-02-22 22:00:00.123456-
06:00)
    --US-military-time               Output timestamp in US military time format (ex: 02-22-2022
22:00:00.123 -06:00)
    --US-time                         Output timestamp in US time format (ex: 02-22-2022 10:00:00.123 PM
-06:00)
    -U, --UTC                        Output time in UTC format (default: local time)

```

json-timeline command examples and config files

The options and config files for `json-timeline` are the same as `csv-timeline` but one extra option `-L, --JSONL-output` for outputting to JSONL format.

level-tuning command

The `level-tuning` command will let you tune the alert levels for rules, either raising or decreasing the risk level as you would like them. This command uses a config file to overwrite the risk levels (the `level` field) of rules in the `rules` folder.

Warning: everytime you run the `update-rules` command, the risk level will be returned back to the original value so you will need to run the `level-tuning` command again afterwards.

Usage: level-tuning [OPTIONS]

Display Settings:

- K, --no-color Disable color output
- q, --quiet Quiet mode: do not display the launch banner

General Options:

- f, --file <FILE> Tune alert levels (default: ./rules/config/level_tuning.txt)
- h, --help Show the help menu

level-tuning command examples

- Normal usage: `hayabusa.exe level-tuning`
- Tune rule alert levels based on your custom config file: `hayabusa.exe level-tuning -f ./config/level_tuning.txt`

level-tuning config file

Hayabusa and Sigma rule authors will estimate the appropriate risk level of the alert when writing their rules. However, sometimes risk levels are not consistent and also the actual risk level may differ according to your environment. Yamato Security provides and maintains a config file at `./rules/config/level_tuning.txt` that you can use to tune your rules as well.

`./rules/config/level_tuning.txt` sample:

```
id,new_level
570ae5ec-33dc-427c-b815-db86228ad43e,informational # 'Application Uninstalled' - Originally
low.
b6ce0b2f-593b-5e1c-e137-d30b2974e30e,high # 'Suspicious Double Extension File Execution' -
Sysmon 1 - Originally critical
452b2159-5e6e-c494-63b9-b385d6195f58,high # 'Suspicious Double Extension File Execution' -
Security 4688 - Originally critical
51ba8477-86a4-6ff0-35fa-7b7f1b1e3f83,high # 'CobaltStrike Service Installations - System' -
System 7045 - Originally critical
daad2203-665f-294c-6d2f-f9272c3214f2,critical # 'Mimikatz DC Sync' - Security 4662 -
Originally high
8b061ac2-31c7-659d-aa1b-36ceed1b03f1,high # 'HackTool - Rubeus Execution' - Sysmon 1 -
Originally critical
be670d5c-31eb-7391-4d2e-d122c89cd5bb,high # 'HackTool - Rubeus Execution' - Security 4688 -
Originally critical
```

In this case, the risk level of the rule with an `id` of `570ae5ec-33dc-427c-b815-db86228ad43e` in the rules directory will have its `level` rewritten to `informational`. The possible levels to set are `critical`, `high`, `medium`, `low` and `informational`.

Warning: The `./rules/config/level_tuning.txt` config file will also be updated to the latest version on the hayabusa-rules repository everytime you run `update-rules`. Therefore, if you make changes to this file, you will loose those changes! If you want to keep a config file for yourself, then create a config file in `./config/level_tuning.txt` and run `hayabusa.exe level-tuning -f ./config/level_tuning.txt`. You can also first do level tuning with the config file provided by Yamato Security and then further tune with your own config file.

list-profiles command

Usage: list-profiles [OPTIONS]

Display Settings:

- K, --no-color Disable color output

```
-q, --quiet      Quiet mode: do not display the launch banner
```

General Options:

```
-h, --help      Show the help menu
```

set-default-profile command

Usage: set-default-profile [OPTIONS]

Display Settings:

```
-K, --no-color  Disable color output
```

```
-q, --quiet      Quiet mode: do not display the launch banner
```

General Options:

```
-h, --help          Show the help menu
```

```
-p, --profile <PROFILE> Specify output profile
```

set-default-profile command examples

- Set the default profile to `minimal` : `hayabusa.exe set-default-profile minimal`
- Set the default profile to `super-verbose` : `hayabusa.exe set-default-profile super-verbose`

update-rules command

The `update-rules` command will sync the `rules` folder with the [Hayabusa rules github repository](#), updating the rules and config files.

Usage: update-rules [OPTIONS]

Display Settings:

```
-K, --no-color  Disable color output
```

```
-q, --quiet      Quiet mode: do not display the launch banner
```

General Options:

```
-h, --help          Show the help menu
```

```
-r, --rules <DIR/FILE> Specify a custom rule directory or file (default: ./rules)
```

update-rules command example

You will normally just execute this: `hayabusa.exe update-rules`

Timeline Output

Output Profiles

Hayabusa has 5 pre-defined output profiles to use in `config/profiles.yaml` :

1. `minimal`
2. `standard` (default)
3. `verbose`
4. `all-field-info`
5. `all-field-info-verbose`
6. `super-verbose`
7. `timesketch-minimal`
8. `timesketch-verbose`

You can easily customize or add your own profiles by editing this file. You can also easily change the default profile with `set-default-profile --profile <profile>` . Use the `list-profiles` command to show the available profiles

and their field information.

1. minimal profile output

%Timestamp%, %Computer%, %Channel%, %EventID%, %Level%, %RecordID%, %RuleTitle%, %Details%

2. standard profile output

%Timestamp%, %Computer%, %Channel%, %EventID%, %Level%, %RecordID%, %RuleTitle%, %Details%, %ExtraFieldInfo% , %RuleID%

3. verbose profile output

%Timestamp%, %Computer%, %Channel%, %EventID%, %Level%, %MitreTactics%, %MitreTags%, %OtherTags%, %RecordID%, %RuleTitle%, %Details%, %ExtraFieldInfo%, %RuleFile%, %RuleID%, %EvtxFile%

4. all-field-info profile output

Instead of outputting the minimal details information, all field information in the EventData and UserData sections will be outputted along with their original field names.

%Timestamp%, %Computer%, %Channel%, %EventID%, %Level%, %RecordID%, %RuleTitle%, %AllFieldInfo%, %RuleFile%, %RuleID%, %EvtxFile%

5. all-field-info-verbose profile output

%Timestamp%, %Computer%, %Channel%, %EventID%, %Level%, %MitreTactics%, %MitreTags%, %OtherTags%, %RecordID%, %RuleTitle%, %AllFieldInfo%, %RuleFile%, %RuleID%, %EvtxFile%

6. super-verbose profile output

%Timestamp%, %Computer%, %Channel%, %EventID%, %Level%, %RuleTitle%, %RuleAuthor%, %RuleModifiedDate%, %Status%, %RecordID%, %Details%, %ExtraFieldInfo%, %MitreTactics%, %MitreTags%, %OtherTags%, %Provider%, %RuleCreationDate%, %RuleFile%, %RuleID%, %EvtxFile%

7. timesketch-minimal profile output

Output to a format compatible with importing into [Timesketch](#).

%Timestamp%, hayabusa, %RuleTitle%, %Computer%, %Channel%, %EventID%, %Level%, %MitreTactics%, %MitreTags%, %OtherTags%, %RecordID%, %Details%, %RuleFile%, %RuleID%, %EvtxFile%

8. timesketch-verbose profile output

%Timestamp%, hayabusa, %RuleTitle%, %Computer%, %Channel%, %EventID%, %Level%, %MitreTactics%, %MitreTags%, %OtherTags%, %RecordID%, %Details%, %ExtraFieldInfo%, %RuleFile%, %RuleID%, %EvtxFile%

Profile Comparison

The following benchmarks were conducted on a 2018 Lenovo P51 (Xeon 4 Core CPU / 64GB RAM) with 3GB of evtx data and 3891 rules enabled. (2023/06/01)

Profile	Processing Time	Output Filesize	Filesize Increase
minimal	8 minutes 50 seconds	770 MB	-30%
standard (default)	9 minutes 00 seconds	1.1 GB	None
verbose	9 minutes 10 seconds	1.3 GB	+20%
all-field-info	9 minutes 3 seconds	1.2 GB	+10%

all-field-info-verbose	9 minutes 10 seconds	1.3 GB	+20%
super-verbose	9 minutes 12 seconds	1.5 GB	+35%

Profile Field Aliases

The following information can be outputted with built-in output profiles:

Alias name	Hayabusa output information
%AllFieldInfo%	All field information.
%Channel%	The name of log. <Event><System><Channel> field.
%Computer%	The <Event><System><Computer> field.
%Details%	The details field in the YML detection rule, however, only hayabusa rules have this field. This field gives extra information about the alert or event and can extract useful data from the fields in event logs. For example, usernames, command line information, process information, etc... When a placeholder points to a field that does not exist or there is an incorrect alias mapping, it will be outputted as n/a (not available). If the details field is not specified (i.e. sigma rules), default details messages to extract fields defined in <code>./rules/config/default_details.txt</code> will be outputted. You can add more default details messages by adding the Provider Name, EventID and details message you want to output in <code>default_details.txt</code> . When no details field is defined in a rule nor in <code>default_details.txt</code> , all fields will be outputted to the details column.
%ExtraFieldInfo%	Print the field information that was not outputted in %Details%.
%EventID%	The <Event><System><EventID> field.
%EvtxFile%	The evtx filename that caused the alert or event.
%Level%	The level field in the YML detection rule. (informational, low, medium, high, critical)
%MitreTactics%	MITRE ATT&CK tactics (Ex: Initial Access, Lateral Movement, etc...).
%MitreTags%	MITRE ATT&CK Group ID, Technique ID and Software ID.
%OtherTags%	Any keyword in the tags field in a YML detection rule which is not included in MitreTactics or MitreTags.
%Provider%	The Name attribute in <Event><System><Provider> field.
%RecordID%	The Event Record ID from <Event><System><EventRecordID> field.
%RuleAuthor%	The author field in the YML detection rule.
%RuleCreationDate%	The date field in the YML detection rule.
%RuleFile%	The filename of the detection rule that generated the alert or event.
%RuleID%	The id field in the YML detection rule.
%RuleModifiedDate%	The modified field in the YML detection rule.
%RuleTitle%	The title field in the YML detection rule.
%Status%	The status field in the YML detection rule.

%Timestamp%	Default is YYYY-MM-DD HH:mm:ss.sss +hh:mm format. <Event><System><TimeCreated SystemTime> field in the event log. The default timezone will be the local timezone but you can change the timezone to UTC with the --UTC option.
-------------	---

Extra Profile Field Alias

You can also add this extra aliases to your output profile if you need it:

Alias name	Hayabusa output information
%RenderedMessage%	The <Event><RenderingInfo><Message> field in WEC forwarded logs.

Note: this is **not** included in any built in profiles so you will need to manually edit the `config/default_profile.yaml` file and add the following line:

```
Message: "%RenderedMessage%"
```

You can also define [event key aliases](#) to output other fields.

Abbreviations

In order to save space, we abbreviate levels, MITRE ATT&CK tactics, channels, providers, field names, etc...

You can turn off some of these abbreviations to see the original channel name, provider name, etc... with the `-b, --disable-abbreviations` option.

Level Abbreviations

In order to save space, we use the following abbreviations when displaying the alert `level`.

- `emer` : emergency
- `crit` : critical
- `high` : high
- `med` : medium
- `low` : low
- `info` : informational
- `undef` : undefined

MITRE ATT&CK Tactics Abbreviations

In order to save space, we use the following abbreviations when displaying MITRE ATT&CK tactic tags. You can freely edit these abbreviations in the `./config/mitre_tactics.txt` configuration file.

- `Recon` : Reconnaissance
- `ResDev` : Resource Development
- `InitAccess` : Initial Access
- `Exec` : Execution
- `Persis` : Persistence
- `PrivEsc` : Privilege Escalation
- `Evas` : Defense Evasion
- `CredAccess` : Credential Access
- `Disc` : Discovery
- `LatMov` : Lateral Movement
- `Collect` : Collection
- `C2` : Command and Control
- `Exfil` : Exfiltration
- `Impact` : Impact

Channel Abbreviations

In order to save space, we use the following abbreviations when displaying Channel. You can freely edit these abbreviations in the `./rules/config/channel_abbreviations.txt` configuration file.

- App : Application
- AppLocker : Microsoft-Windows-AppLocker/*
- BitsCli : Microsoft-Windows-Bits-Client/Operational
- CodeInteg : Microsoft-Windows-CodeIntegrity/Operational
- Defender : Microsoft-Windows-Windows Defender/Operational
- DHCP-Svr : Microsoft-Windows-DHCP-Server/Operational
- DNS-Svr : DNS Server
- DvrFmwk : Microsoft-Windows-DriverFrameworks-UserMode/Operational
- Exchange : MExchange Management
- Firewall : Microsoft-Windows-Windows Firewall With Advanced Security/Firewall
- KeyMgtSvc : Key Management Service
- LDAP-Cli : Microsoft-Windows-LDAP-Client/Debug
- NTLM : Microsoft-Windows-NTLM/Operational
- OpenSSH : OpenSSH/Operational
- PrintAdm : Microsoft-Windows-PrintService/Admin
- PrintOp : Microsoft-Windows-PrintService/Operational
- PwSh : Microsoft-Windows-PowerShell/Operational
- PwShClassic : Windows PowerShell
- RDP-Client : Microsoft-Windows-TerminalServices-RDPClient/Operational
- Sec : Security
- SecMitig : Microsoft-Windows-Security-Mitigations/*
- SmbCliSec : Microsoft-Windows-SmbClient/Security
- SvcBusCli : Microsoft-ServiceBus-Client
- Sys : System
- Sysmon : Microsoft-Windows-Sysmon/Operational
- TaskSch : Microsoft-Windows-TaskScheduler/Operational
- WinRM : Microsoft-Windows-WinRM/Operational
- WMI : Microsoft-Windows-WMI-Activity/Operational

Other Abbreviations

The following abbreviations are used in rules in order to make the output as concise as possible:

- Acct -> Account
- Addr -> Address
- Auth -> Authentication
- Cli -> Client
- Chan -> Channel
- Cmd -> Command
- Cnt -> Count
- Comp -> Computer
- Conn -> Connection/Connected
- Creds -> Credentials
- Crit -> Critical
- Disconn -> Disconnection/Disconnected
- Dir -> Directory
- Drv -> Driver
- Dst -> Destination
- EID -> Event ID
- Err -> Error
- Exec -> Execution

- `FW` -> Firewall
- `Grp` -> Group
- `Img` -> Image
- `Inj` -> Injection
- `Krb` -> Kerberos
- `LID` -> Logon ID
- `Med` -> Medium
- `Net` -> Network
- `Obj` -> Object
- `Op` -> Operational/Operation
- `Proto` -> Protocol
- `PW` -> Password
- `Reconn` -> Reconnection
- `Req` -> Request
- `Rsp` -> Response
- `Sess` -> Session
- `Sig` -> Signature
- `Susp` -> Suspicious
- `Src` -> Source
- `Svc` -> Service
- `Svr` -> Server
- `Temp` -> Temporary
- `Term` -> Termination/Terminated
- `Tkt` -> Ticket
- `Tgt` -> Target
- `Unkwn` -> Unknown
- `Usr` -> User
- `Perm` -> Permanent
- `Pkg` -> Package
- `Priv` -> Privilege
- `Proc` -> Process
- `PID` -> Process ID
- `PGUID` -> Process GUID (Global Unique ID)
- `Ver` -> Version

Progress Bar

The progress bar will only work with multiple evtx files. It will display in real time the number and percent of evtx files that it has finished analyzing.

Color Output

The alerts will be outputted in color based on the alert `level`. You can change the default colors in the config file at `./config/level_color.txt` in the format of `level,(RGB 6-digit ColorHex)`. If you want to disable color output, you can use `-K, --no-color` option.

Results Summary

Total events, the number of events with hits, data reduction metrics, total and unique detections, dates with the most detections, top computers with detections and top alerts are displayed after every scan.

Detection Fequency Timeline

If you add the `-T, --visualize-timeline` option, the Event Frequency Timeline feature displays a sparkline frequency timeline of detected events. Note: There needs to be more than 5 events. Also, the characters will not render

correctly on the default Command Prompt or PowerShell Prompt, so please use a terminal like Windows Terminal, iTerm2, etc...

Hayabusa Rules

Hayabusa detection rules are written in a sigma-like YAML format and are located in the `rules` folder. The rules are hosted at <https://github.com/Yamato-Security/hayabusa-rules> so please send any issues and pull requests for rules there instead of the main Hayabusa repository.

Please read [the hayabusa-rules repository README](#) to understand about the rule format and how to create rules.

All of the rules from the hayabusa-rules repository should be placed in the `rules` folder. `informational` level rules are considered `events`, while anything with a `level` of `low` and higher are considered `alerts`.

The hayabusa rule directory structure is separated into 2 directories:

- `builtin` : logs that can be generated by Windows built-in functionality.
- `sysmon` : logs that are generated by [sysmon](#).

Rules are further separated into directories by log type (Example: Security, System, etc...) and are named in the following format:

Please check out the current rules to use as a template in creating new ones or for checking the detection logic.

Sigma v.s. Hayabusa (Built-in Sigma Compatible) Rules

Hayabusa supports Sigma rules natively with a single exception of handling the `logsource` fields internally. In order to reduce false positives, Sigma rules should be run through our convertor explained [here](#). This will add the proper `Channel` and `EventID`, and perform field mapping for certain categories like `process_creation`.

Almost all Hayabusa rules are compatible with the Sigma format so you can use them just like Sigma rules to convert to other SIEM formats. Hayabusa rules are designed solely for Windows event log analysis and have the following benefits:

1. An extra `details` field to display additional information taken from only the useful fields in the log.
2. They are all tested against sample logs and are known to work.
3. Extra aggregators not found in sigma, such as `|equalsfield` and `|endswithfield`.

To our knowledge, hayabusa provides the greatest native support for sigma rules out of any open source Windows event log analysis tool.

Other Windows Event Log Analyzers and Related Resources

- [APT-Hunter](#) - Attack detection tool written in Python.
- [Awesome Event IDs](#) - Collection of Event ID resources useful for Digital Forensics and Incident Response
- [Chainsaw](#) - Another sigma-based attack detection tool written in Rust.
- [DeepBlueCLI](#) - Attack detection tool written in Powershell by [Eric Conrad](#).
- [Epagneul](#) - Graph visualization for Windows event logs.
- [EventList](#) - Map security baseline event IDs to MITRE ATT&CK by [Miriam Wiesner](#).
- [Mapping MITRE ATT&CK with Window Event Log IDs](#) - by [Michel de CREVOISIER](#)
- [EvtxECmd](#) - Evtx parser by [Eric Zimmerman](#).
- [EVTXtract](#) - Recover EVTX log files from unallocated space and memory images.
- [EvtxToElk](#) - Python tool to send Evtx data to Elastic Stack.
- [EVTX ATTACK Samples](#) - EVTX attack sample event log files by [SBousseaden](#).
- [EVTX-to-MITRE-Attack](#) - EVTX attack sample event log files mapped to ATT&CK by [Michel de CREVOISIER](#)
- [EVTX parser](#) - the Rust evtx library we use written by [@OBenamram](#).
- [Grafiki](#) - Sysmon and PowerShell log visualizer.
- [LogonTracer](#) - A graphical interface to visualize logons to detect lateral movement by [JPCERTCC](#).
- [NSA Windows Event Monitoring Guidance](#) - NSA's guide on what to monitor for.

- [RustyBlue](#) - Rust port of DeepBlueCLI by Yamato Security.
- [Sigma](#) - Community based generic SIEM rules.
- [SOF-ELK](#) - A pre-packaged VM with Elastic Stack to import data for DFIR analysis by [Phil Hagen](#)
- [so-import-evtx](#) - Import evtx files into Security Onion.
- [SysmonTools](#) - Configuration and off-line log visualization tool for Sysmon.
- [Timeline Explorer](#) - The best CSV timeline analyzer by [Eric Zimmerman](#).
- [Windows Event Log Analysis - Analyst Reference](#) - by Forward Defense's Steve Anson.
- [Zircolite](#) - Sigma-based attack detection tool written in Python.

Windows Logging Recommendations

In order to properly detect malicious activity on Windows machines, you will need to improve the default log settings. We have created a separate project to document what log settings need to be enabled as well as scripts to automatically enable the proper settings at <https://github.com/Yamato-Security/EnableWindowsLogSettings>.

We also recommend the following sites for guidance:

- [JSCU-NL \(Joint Sigint Cyber Unit Netherlands\) Logging Essentials](#)
- [ACSC \(Australian Cyber Security Centre\) Logging and Forwarding Guide](#)
- [Malware Archaeology Cheat Sheets](#)

Sysmon Related Projects

To create the most forensic evidence and detect with the highest accuracy, you need to install sysmon. We recommend the following sites and config files:

- [TrustedSec Sysmon Community Guide](#)
- [Sysmon Modular](#)
- [SwiftOnSecurity Sysmon Config](#)
- [SwiftOnSecurity Sysmon Config fork by Neo23x0](#)
- [SwiftOnSecurity Sysmon Config fork by ion-storm](#)

Community Documentation

English

- 2023/12/11 [Unleashing the Hayabusa Feathers: My Top Features Revealed!](#) by Christian Henriksen
- 2023/10/16 [Incident response and threat hunting using hayabusa tool](#) by Md. Mahim Bin Firoj
- 2023/03/21 [Find Threats in Event Logs with Hayabusa](#) by [Eric Capuano](#)
- 2023/03/14 [Rust Performance Guide for Hayabusa Developers](#) by Fukusuke Takahashi
- 2022/06/19 [Velociraptor Walkthrough and Hayabusa Integration](#) by [Eric Capuano](#)
- 2022/01/24 [Graphing Hayabusa results in neo4j](#) by Matthew Seyer ([@forensic_matt](#))

Japanese

- 2024/01/24 [LME x Hayabusa Windows イベントログの と の](#) by NEC Security Blog
- 2023/09/29 [Fast Forensics with Hayabusa and Splunk](#) by NEC Security Blog
- 2023/09/13 [Windows Event Log Analysis with Hayabusa](#) by FFRI
- 2022/03/14 [Rust Performance Guide for Hayabusa Developers](#) by Fukusuke Takahashi
- 2022/01/22 [Visualizing Hayabusa results in Elastic Stack](#) by [@kzzzo2](#)
- 2021/12/31 [Intro to Hayabusa](#) by itiB ([@itiB_S144](#))
- 2021/12/27 [Hayabusa internals](#) by Kazuminn ([@k47_um1n](#))

Contribution

We would love any form of contribution. Pull requests, rule creation and sample evtx logs are the best but feature requests, notifying us of bugs, etc... are also very welcome.

At the least, if you like our tool then please give us a star on GitHub and show your support!

Bug Submission

Please submit any bugs you find [here](#). This project is currently actively maintained and we are happy to fix any bugs reported.

If you find any issues (false positives, bugs, etc...) with Hayabusa rules, please report them to the hayabusa-rules GitHub issues page [here](#).

If you find any issues (false positives, bugs, etc...) with Sigma rules, please report them to the upstream SigmaHQ GitHub issues page [here](#).

License

Hayabusa is released under [AGPLv3](#) and all rules are released under the [Detection Rule License \(DRL\) 1.1](#). You may freely use Hayabusa internally, SaaS solutions, for consulting work, etc... However, if you use Hayabusa in a type of SaaS solution and make improvements to it, we ask you to open-source those improvements and give back to the project.

Hayabusa uses GeoLite2 data created by MaxMind, available from <https://www.maxmind.com>.

Twitter

You can receive the latest news about Hayabusa, rule updates, other Yamato Security tools, etc... by following us on Twitter at [@SecurityYamato](#).